队伍编号	MC2600266
题号	B

基于动作特征表征与鲁棒动态规划的 PM01 机器人竞技策略优化研究

摘 要

针对 PM01 人形机器人在格斗竞技场景中的动作选择、攻防匹配、单场策略优化与 BO3 赛制下资源调度问题，本文构建了一套由动作特征表征、综合评分、鲁棒动态规划与资源递推组成的统一建模框架。首先，依据 PM01 官网产品页中“部署代码仓库”入口所指向的 GitHub 开源仓库提取并整理得到的整机质量、肢段长度、支撑尺寸、代表关节角速度上限与关节力矩上限等关键参数，建立 13 类攻击动作的结构化特征表征，分别量化动力学收益、平衡扰动、主动补偿、能耗代价与战术可用性，并据此形成攻击动作综合效能评估模型。结果表明，直拳、勾拳、摆拳和组合拳在打击收益、姿态稳定与比赛可用性之间形成最优平衡，可作为主攻动作集。

在此基础上，本文进一步构造 22 类防守动作特征表征，从封线能力、脱线能力、机械缓冲、平衡储备、时序匹配与反击窗口等方面建立攻防匹配损失模型，得到 13 类进攻动作对应的最优防守映射。结果显示，拳类攻击对应单手拍挡、肘挡与护头防御形成稳定低损失带，冲撞更适合卸力缓冲与重心补偿，回旋踢更适合左右侧闪等脱线策略。随后，针对单场比赛的连续决策问题，本文将比赛状态离散为机体完好度、平衡风险与距离状态构成的有限状态空间，构建混合最坏响应与平均响应的鲁棒动态规划模型，求得不同状态下的最优动作。模型表明，PM01 单场最优策略应呈现“低风险主动压制、高风险快速恢复”的阈值切换结构。最后，在 BO3 赛制下引入故障等级、能量状态与资源余量，建立资源决策动态规划模型。结果表明，战术暂停具有最高全局边际价值，人工复位适用于倒地临界状态，紧急维修应保留给中高等级故障情形。

本文形成了从动作评估、攻防匹配到赛中策略与赛制级资源调度的完整链条，并在此基础上提出具身智能产业标准、安全闭环、仿真训练、辅助决策与场景化试点等建议，可为机器人竞技建模与高动态具身系统决策提供参考。

关键词：人形机器人竞技；攻防动作特征表征；鲁棒动态规划；资源优化决策；具身智能

目录

一、 问题提出与分析 1
1.1 问题背景 1
1.2 问题重述 1
1.3 问题分析 1
二、 模型假设与合理性说明 2
三、 符号说明 2
四、 数据说明 3
五、 模型建立与求解 4
5.1 问题一：攻击动作综合效能评估 4
5.1.1 问题引入 4
5.1.2 模型构建 5
5.1.3 求解方法 6
5.1.4 结果解析 6
5.2 问题二：攻防匹配优化 9
5.2.1 问题引入 9
5.2.2 模型构建 9
5.2.3 求解方法 11
5.2.4 结果解析 11
5.3 问题三：单场竞技赛策略优化 14
5.3.1 问题引入 14
5.3.2 模型构建 14
5.3.3 求解方法 15
5.3.4 结果解析 15
5.4 问题四：BO3 赛制下资源使用决策优化 18
5.4.1 问题引入 18
5.4.2 模型构建 18
5.4.3 求解方法 19
5.4.4 结果解析 19
5.5 问题五：面向具身智能产业发展的建议 23
5.5.1 建议形成依据 23
5.5.2 建议书正文 23
参考文献 24
附录：核心求解代码 26

一、问题提出与分析

1.1 问题背景

题目围绕 PM01 人形机器人在竞技格斗场景中的动作选择与对抗决策展开，要求在给定机器人结构参数、动作类型和赛制规则的条件下，从单动作评价逐步扩展到攻防匹配、单场比赛策略和 BO3 系列赛资源调度。与传统静态打分问题不同，本题同时涉及动作动力学、平衡控制、战术可行性、状态转移与资源约束，是一个典型的“机器人运动能力评估 + 对抗决策优化”综合建模问题。

在机器人竞技场景中，“能打”并不等于“好用”。某些动作瞬时冲击大，但支撑切换复杂、恢复时间长、失衡风险高；某些防守动作覆盖能力强，但若执行后平衡储备不足，同样会在连续对抗中放大风险。因此，本题的核心不只是对单个指标排序，而是要建立一套兼顾打击收益、姿态稳定、连续攻防衔接与赛制资源约束的统一模型。

1.2 问题重述

结合题意，本文需要依次回答以下五个问题：

1. 在给定的 13 类攻击动作中，综合比较各动作的攻击效果、稳定性与比赛可用性，确定 PM01 的主攻动作集及全部动作优先级。

2. 面对 13 类已知来袭攻击，为 PM01 设计合理的防守动作特征表征，并从 22 类防守动作中为每一类进攻动作匹配最优防守方案。

3. 在忽略机器人故障的条件下，建立单场比赛中的动态对抗策略模型，给出不同状态下的最优攻防动作选择与比赛方案。

4. 在 BO3 赛制下进一步考虑故障风险、能量状态和有限资源约束，优化战术暂停、人工复位和紧急维修的使用时机与使用强度。

5. 基于前四问的定量结果，提出面向具身智能产业发展的建议，使建模结果能够从竞技场景延展到更广泛的实际应用场景。

1.3 问题分析

五个问题之间具有明显的递进关系。问题一解决“哪些动作值得优先使用”的静态评价问题，是后续模型的基础；问题二解决“面对某种攻击时如何防守”的双动作匹配问题，为连续博弈提供攻防映射；问题三将前两问得到的动作特征表征、最优攻击与最优防守结果统一放入单场比赛状态转移框架，求解随状态变化的最优动作；问题四则在问题三的基础上引入故障与资源状态，将局内最优扩展为 BO3 全局最优；问题五则要求把前四问中的技术结论抽象为工程建议。

从建模难点看，本题至少包含四层挑战：一是如何把机器人几何、质量、速度、支撑方式和平衡补偿等参数转化为可计算的动作特征表征；二是如何同时刻画“攻击收益”

和“稳定风险”，避免高冲击但高失衡的动作被错误高估；三是如何把单动作结果转化为连续比赛中的状态反馈策略；四是如何在 BO3 赛制下处理故障、能量和资源之间的时序权衡。因此，本文采用“参数驱动动作特征表征 + 攻防损失建模 + 鲁棒动态规划 + 系列赛资源递推”的总体思路，使静态能力评估与动态决策求解形成一条完整链路。

二、模型假设与合理性说明

为保证模型可计算、可解释且与竞技场景一致，作如下假设：

1. 左右对称动作视为同类动作处理，例如左直拳与右直拳统一记为直拳，左侧闪与右侧闪统一归入侧闪类动作。这一假设可以降低动作空间维数，同时不影响高层战术判断。

2. 所有动作均从标准站立准备姿态起始，比较的是标准执行版本，而非临场极端超常发挥或严重失误。该假设使问题聚焦于动作本身，而不是操作者随机波动。

3. 动作特征表征中的时长、力矩利用率、单脚支撑比例、质心位移和补偿姿态等参数，均代表该动作在 PM01 平台上的可执行标称值，用于高层策略建模，而不追求底层控制器级别的毫秒精度。

4. 单场比赛中的机体完好度、平衡风险、距离状态、能量等级和故障等级采用有限离散状态表示，用于刻画比赛中的主要趋势。该离散化能保留关键物理意义，并使动态规划可以求解。

5. 对手与我方共享同一组高层动作集合，但在问题三中采用“最坏响应 + 平均响应”的混合鲁棒准则建模，以反映真实比赛中既存在强反制，也存在非极端策略选择。

6. 三维仿真用于验证动作轨迹和姿态变化的合理性，不等同于全刚体接触动力学仿真；其结论用于支持动作排序与防守姿态解释，而非直接替代真实比赛测试。

上述假设并未改变题目本质，而是把原始复杂的连续机器人控制问题抽象为高层策略优化问题，使模型既保留主要运动学与稳定性信息，又能够形成清晰的求解链条。

三、符号说明

符号	符号说明
M	PM01 机器人整机质量
$l_{\mathrm{arm}}, l_{\mathrm{leg}}, l_{\mathrm{torso}}$	机器人手臂总长、腿部总长与躯干长度
b_{foot}	足宽，用于支撑裕度归一化
S_a^{dyn}	动作 a 的动力学收益
D_a, C_a, R_a	动作 a 的稳定扰动指数、补偿指数与平衡储备
$\widehat{W}_a, Q(a,s)$	动作 a 的归一化能耗代价与对部位 s 的综合评分

符号	符号说明
$I_{a,d}, L(a,d)$	攻击动作 a 与防守动作 d 的拦截得分和总损失
$s^\star(a), d^\star(a)$	动作 a 的最优攻击部位与来袭动作 a 对应的最优防守动作
s_t	问题三单场比赛状态
$h_t^{\text{self}}, h_t^{\text{opp}}$	我方与对手机体完好度
$b_t^{\text{self}}, b_t^{\text{opp}}$	我方与对手平衡风险
d_t	双方距离状态
$\mathcal{H}, \mathcal{B}, \mathcal{D}_r$	问题三中的机体完好度集合、平衡风险集合与距离状态集合
$\mathcal{A}, \mathcal{D}, \mathcal{U}, \mathcal{U}(s_t)$	攻击动作集、防守动作集、总动作集与状态相关可行动作集
u_t, v_t	问题三中我方与对手在第 t 步选择的动作
$G_t^{\text{out}}, G_t^{\text{in}}$	第 t 步我方输出伤害与承受伤害
$R_t(s_t,u_t,v_t), C_E(u_t)$	单步收益函数与动作能量代价项
$V_t(s)$	问题三的价值函数
$Q_t(s,u,v), f(s,u,v)$	问题三的状态动作值函数与状态转移函数
$\Pi_{\mathcal{B}}, \gamma, \beta$	平衡风险投影算子、折扣因子与鲁棒系数
x_t	问题四 BO3 资源决策状态
w_t, l_t, k_t, a_t	系列赛已胜局数、已负局数、局内资源决策步与局内优势
ϕ_t, e_t, b_t	故障等级、能量等级与问题四中的平衡风险
$p_t, r_t, m_t, \Omega(x_t)$	战术暂停、人工复位、紧急维修的剩余次数与可选资源动作集
A_q, D_q	问题一主攻动作强度汇总值与问题二防守鲁棒性汇总值
$V_0^{(3)}, \rho_w^{(3)}, V_{\text{press}}^{(3)}$	问题三初始状态价值、胜势状态比例与最坏响应压力测试价值
$\chi, E(x_t)$	由前三问结果形成的战术基础偏置与综合攻防边际
$P_\phi(x_t), P_{\text{break}}(x_t)$	故障发生或恶化概率与本局提前失利概率
$F(x)$	问题四中状态 x 下最终赢得 BO3 的最大概率
$\pi(x), x^{\text{win}}, x^{\text{lose}}$	当前局取胜概率以及本局胜负结束后的后继状态
M_i, R_i, G_i, C_i	问题五中建议 i 的影响度、紧迫度、可行性与协同性
P_i	建议 i 的综合优先级指标

四、数据说明

本文建模所使用的数据主要来自四个层次：第一，题目给出的赛制规则、资源约束与比赛流程；第二，从 PM01 官网产品页“部署代码仓库”入口所指向的 GitHub 开源仓库中提取并整理得到的整机质量、肢段长度、足部支撑尺寸、代表角速度和关节力矩上限等基础参数；第三，依据上述机器人参数手工标定并校准的 13 类攻击动作特征表征与 22 类防守动作特征表征；第四，在问题一和问题二基础上进一步生成的攻击优先级、防守损失矩阵、状态价值、鲁棒性统计和资源使用结果等派生数据。这些数据共同支撑了从单动作评价到 BO3 系列赛优化的连续建模。

问题一与问题二最重要的机器人基础参数如表2 所示。它们决定了动作半径、速度

上限、机械能耗尺度和支撑裕度的计算基准，是全文最核心的底层参数输入。

表 2 PM01 关键建模参数

参数类别	参数名称	数值	作用
整机参数	总质量 M	40.9208 kg	计算有效质量、线动量与动能
上肢几何	手臂总长 l_{arm}	0.3716 m	计算拳法标称打击速度
下肢几何	腿部总长 l_{leg}	0.7183 m	计算腿法标称打击速度
躯干几何	躯干长度 l_{torso}	0.3589 m	构造特殊动作等效半径
支撑参数	足宽 b_{foot}	0.1071 m	归一化支撑裕度
执行器参数	轻载关节力矩上限	61 N · m	估计上肢动作能耗
执行器参数	重载关节力矩上限	164 N · m	估计下肢动作能耗
速度参数	上肢代表角速度上限	35.20 rad/s	计算拳法标称打击速度
速度参数	下肢代表角速度上限	29.27 rad/s	计算腿法标称打击速度

除上述基础参数外，本文还构造了面向建模的数据层：问题一使用 13 类攻击动作的结构化特征表征及三维仿真结果；问题二使用 22 类防守动作特征表征与 13×22 攻防损失矩阵；问题三建立了由机体完好度、平衡与距离组成的 1200 个离散状态，并以其中 432 个非终止状态进行策略优化；问题四在 BO3 维度上增加了局分、能量、故障与资源状态，构建系列赛级动态规划输入。这些数据既包含机器人本体信息，也包含由前序问题逐步生成的结果数据，因此能够支撑全文的一体化建模。

五、模型建立与求解

5.1 问题一：攻击动作综合效能评估

5.1.1 问题引入

问题一的目标是在 13 类攻击动作中筛选出更适合 PM01 机器人的主攻动作，并给出全部动作的综合优先级。由于机器人竞技中的动作优劣不仅取决于瞬时冲击大小，还受到姿态稳定、支撑切换、能耗代价和比赛可用性的共同影响，因此本文不采用单一冲击指标排序，而是从“动力学收益 + 稳定性约束 + 战术可用性”三个方面建立综合评价模型。

5.1.2 模型构建

为统一表示不同攻击动作的结构特征，设动作 a 的特征向量为

$$\mathrm{x}_a = \left[\mathrm{x}_a^{\mathrm{atk}}, \mathrm{x}_a^{\mathrm{bal}}, \mathrm{x}_a^{\mathrm{cmp}}, \mathrm{x}_a^{\mathrm{tar}}\right], \tag{1}$$

其中

$$\mathrm{x}_a^{\mathrm{atk}} = (\rho_m, u_v, n_{25}, n_{90}, u_\tau, \phi_{25}, \phi_{90}, t_a), \tag{2}$$

$$\mathrm{x}_a^{\mathrm{bal}} = (\rho_Z, \rho_L, \rho_h, \xi_a, \rho_{ss}, m_{\mathrm{mode}}), \tag{3}$$

$$\mathrm{x}_a^{\mathrm{cmp}} = \left(\Delta z_{\mathrm{drop}}, m_{\mathrm{margin}}, \varphi^{\mathrm{torso}}, \varphi^{\mathrm{arm}}, \kappa_{\mathrm{knee}}\right), \tag{4}$$

$$\mathrm{x}_a^{\mathrm{tar}} = \left(\eta_{a,H}, \eta_{a,T}, \eta_{a,L}\right). \tag{5}$$

上述特征向量分别描述动作的攻击效率、平衡扰动、主动补偿与命中能力。

根据 PM01 上肢、下肢与躯干长度以及代表角速度上限，定义不同类别动作的标称打击速度

$$v_{\mathrm{ref}}^{\mathrm{punch}} = 0.45\,\bar{\omega}_u\, l_{\mathrm{arm}}, \tag{6}$$

$$v_{\mathrm{ref}}^{\mathrm{kick}} = 0.38\,\bar{\omega}_l\, l_{\mathrm{leg}}, \tag{7}$$

$$v_{\mathrm{ref}}^{\mathrm{combo}} = \frac{1}{2}\left(v_{\mathrm{ref}}^{\mathrm{punch}} + v_{\mathrm{ref}}^{\mathrm{kick}}\right), \tag{8}$$

$$v_{\mathrm{ref}}^{\mathrm{special}} = 0.30\left(\bar{\omega}_l l_{\mathrm{leg}} + \omega_w l_{\mathrm{torso}}\right). \tag{9}$$

于是动作 a 的实际打击速度为

$$v_a = u_v(a)\, v_{\mathrm{ref}}^{(c(a))}, \tag{10}$$

有效质量定义为

$$m_a = \rho_m(a) M. \tag{11}$$

进一步可得动作的线动量、动能与等效角动量

$$p_a = m_a v_a, \tag{12}$$

$$E_a = \frac{1}{2} m_a v_a^2, \tag{13}$$

$$L_a = I_a \omega_a, \qquad I_a = 0.35 m_a r_a^2, \qquad \omega_a = \frac{v_a}{r_a}. \tag{14}$$

据此构造动力学收益

$$S_a^{\mathrm{dyn}} = \left(0.34\widehat{p}_a + 0.41\widehat{E}_a + 0.25\widehat{L}_a\right)(1 + 0.05\ln h_a), \tag{15}$$

其中 h_a 为有效命中次数。

为刻画稳定性约束，定义稳定扰动指数

$$D_a = 0.30\rho_Z(a) + 0.24\rho_L(a) + 0.12\rho_h(a) + 0.16\rho_{ss}(a) + 0.10\xi_a + \delta\big(m_{\mathrm{mode}}(a)\big), \tag{16}$$

以及主动补偿指数

$$C_a = 0.22\frac{m_{\text{margin},a}}{b_{\text{foot}}} + 0.18\frac{\Delta z_{\text{drop},a}}{l_{\text{leg}}} + 0.18\kappa_{\text{knee},a} + 0.10\varphi_a^{\text{torso}} + 0.10\varphi_a^{\text{arm}}, \tag{17}$$

并利用动作特征表征中的稳定得分构造平衡储备

$$R_a = 0.60\sigma_a + 0.40 \cdot \text{clip}\Big(0.55 + 1.10(\widehat{C}_a - \widehat{D}_a), 0, 1\Big). \tag{18}$$

同时，以“关节力矩 × 行程 × 时长”的近似形式定义能耗代价

$$W_a = (n_{25}\tau_{25}\phi_{25} + n_{90}\tau_{90}\phi_{90})\, u_\tau(a)\,(1 + 0.30u_v(a))\, t_a, \tag{19}$$

归一化后记为 $\widehat{W}_a$。

进一步考虑目标部位差异，定义动作对部位 s 的部位收益

$$S_{a,s}^{\text{site}} = S_a^{\text{dyn}} \cdot \omega_s^{\text{dmg}} \cdot \omega_s^{\text{exp}} \cdot \eta_{a,s}, \tag{20}$$

最终得到综合评分函数

$$Q(a,s) = \eta_a^{\text{opp}}\left[0.50S_{a,s}^{\text{site}} + 0.34R_a - 0.16\widehat{W}_a - 0.22(1 - R_a) - P_s^{\text{height}} - P_a^{\text{support}}\right]. \tag{21}$$

5.1.3 求解方法

对任意动作 a，先在头部、躯干和四肢三个部位中求取最优攻击部位

$$s^\star(a) = \arg\max_{s\in\{H,T,L\}} Q(a,s), \qquad Q^\star(a) = Q\big(a, s^\star(a)\big). \tag{22}$$

随后按照 $Q^\star(a)$ 由大到小进行排序，即得到全部攻击动作的综合优先级。为检验排序结果的稳定性，本文又进行了两类辅助求解：一是利用稳定性约束动作特征表征驱动 13 个动作的三维运动学仿真，观察主要发力部件的轨迹与姿态变化；二是对综合评分中的主要权重进行随机扰动，统计每个动作进入 Top4 的频率，从而验证排序结果是否对个别经验权重过于敏感。

5.1.4 结果解析

问题一的综合排序结果如表3 所示。

表 3 13 类攻击动作综合排序结果

排名	动作	最优部位	$Q^\star$	R_a	$\widehat{W}_a$	η_a
1	直拳	躯干	0.355	0.970	0.019	0.98
2	勾拳	躯干	0.343	0.970	0.033	0.95
3	摆拳	躯干	0.315	0.945	0.088	0.90
4	组合拳	躯干	0.222	0.931	0.081	0.92

排名	动作	最优部位	$Q^{\star}$	R_a	$\widehat{W}_a$	η_a
5	冲撞	躯干	0.182	0.557	0.291	0.60
6	前踢	躯干	0.114	0.669	0.191	0.84
7	膝撞	躯干	0.114	0.691	0.122	0.80
8	侧踢	躯干	0.078	0.615	0.284	0.78
9	倒地反击	躯干	0.065	0.853	0.431	0.18
10	低扫腿	四肢	0.047	0.679	0.237	0.74
11	拳腿组合	躯干	0.039	0.632	0.511	0.76
12	回旋踢/转身踢	躯干	-0.001	0.540	0.565	0.62
13	五连踢	躯干	-0.102	0.506	1.000	0.48

从结果看，前四名全部为拳法动作，说明在 PM01 的尺寸、速度与支撑条件下，上肢动作在收益、稳定和可用性之间形成最优平衡。冲撞虽然具备较高的基础冲击能力，但由于姿态风险较大、战术可用性受限，最终只能排在第 5 位。回旋踢和五连踢并非打击不足，而是在支撑切换复杂、能耗高和恢复要求高的多重影响下综合得分明显下降。

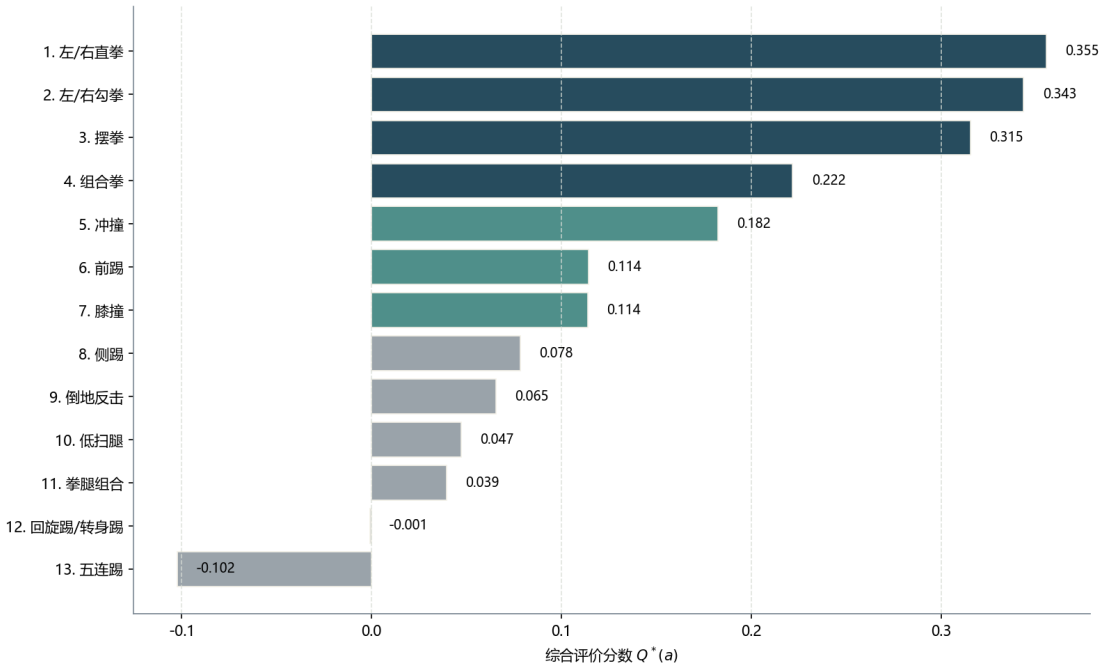

图 1　13 类攻击动作综合评分排序

结合综合评分和平衡储备，可将直拳、勾拳、摆拳和组合拳定义为主攻动作集，将冲撞、前踢和膝撞定义为情景动作集。与此同时，在全部 13 类动作中，有 12 类动作的最优攻击部位为躯干，仅低扫腿以四肢为最优目标，说明躯干是题目比赛语境下综合性价比最高的主要攻击区域。

为验证动作特征表征与排序结果的执行合理性，本文利用稳定性约束动作特征表征驱动 13 个动作进行三维运动学仿真，总览如图2 所示。图中黄色轨迹线表示主要发力

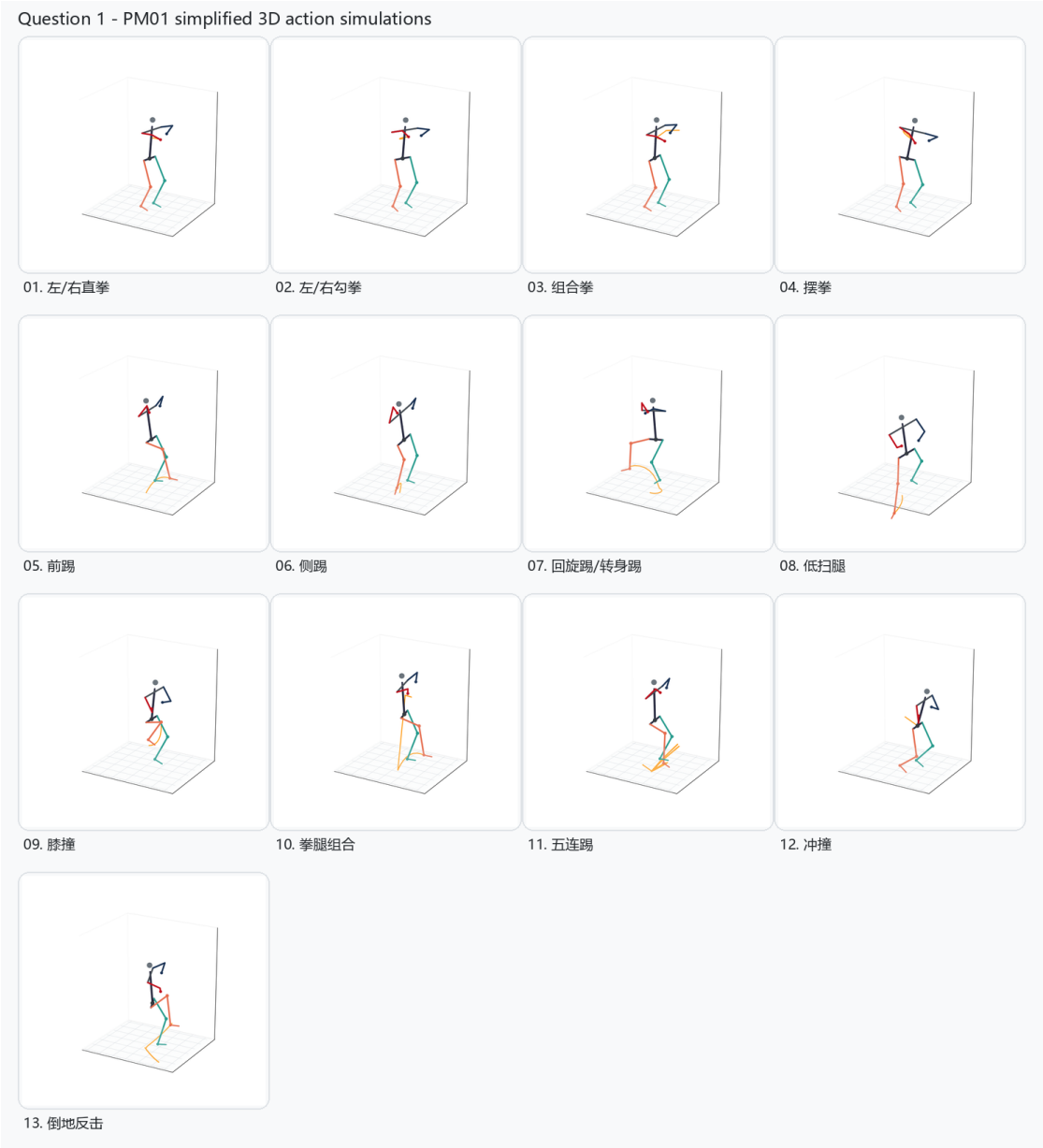

图 2 13 类攻击动作三维运动学仿真总览

部件的运动轨迹：拳法动作对应手腕轨迹，腿法动作对应踝关节或膝部轨迹，冲撞动作对应骨盆前移轨迹。仿真显示，直拳、勾拳、摆拳和组合拳轨迹紧凑、支撑变化小；侧踢、回旋踢和五连踢表现出更明显的横向质心转移和支撑切换；冲撞虽有明显前倾，但可通过屈膝与步点调节维持一定稳定性。这与综合排序结果相互印证。

进一步对式 (21) 式中收益、平衡、能耗和风险权重进行随机扰动，统计动作进入 Top4 的频率，结果如图3 所示。结果表明，直拳、勾拳、摆拳和组合拳进入 Top4 的频率均为 1.0，说明主攻动作集在合理权重扰动范围内保持不变，模型结论具有较强稳定性。

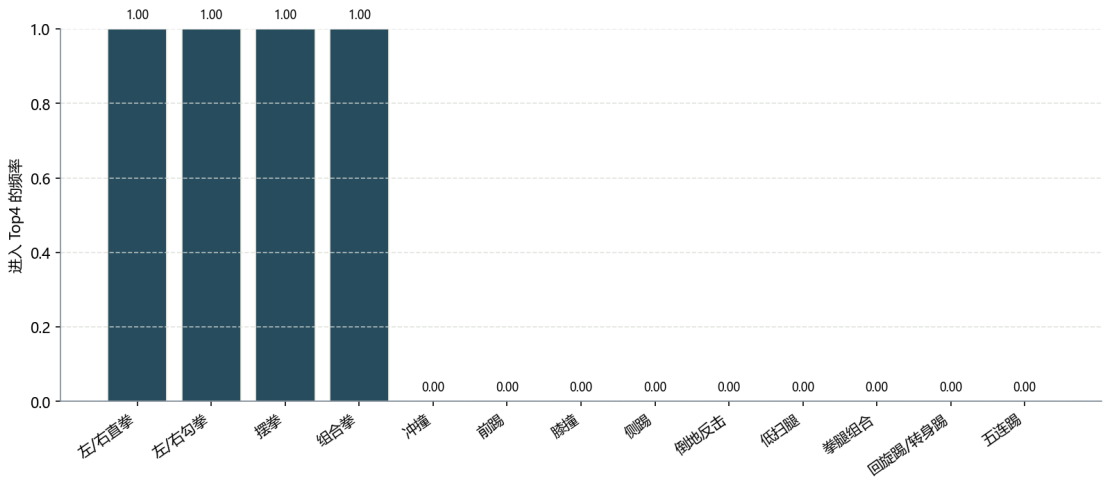

图 3 评分权重扰动下动作进入 Top4 的频率

5.2 问题二：攻防匹配优化

5.2.1 问题引入

问题二不再单独评价“哪一种进攻更强”，而是基于问题一得到的攻击动作特征表征与最优攻击部位，进一步寻找每一种来袭动作对应的最优防守策略。与问题一相比，问题二更强调三个方面：其一，防守动作是否真正覆盖或脱离来袭轨迹；其二，完成防守后机器人能否维持稳定站姿；其三，面对组合攻击时，防守动作是否来得及恢复并衔接下一拍。因此，问题二的核心是建立一个同时刻画“挡得住、躲得开、站得稳、跟得上”的攻防匹配模型。

5.2.2 模型构建

设防守动作 d 的特征向量为

$$\mathrm{y}_d = \left[\mathrm{y}_d^{\mathrm{cov}}, \mathrm{y}_d^{\mathrm{eva}}, \mathrm{y}_d^{\mathrm{bal}}, \mathrm{y}_d^{\mathrm{tim}}, \mathrm{y}_d^{\mathrm{eng}}\right], \tag{23}$$

其中各分量分别描述覆盖封线能力、闪避脱线能力、平衡控制能力、时序恢复能力与机械能耗。进一步根据功能机理可将 22 类防守动作划分为格挡封线类、闪避脱线类、姿态压缩类、平衡缓冲类、地面过渡类和复合反制类六组，如表4 所示。

首先，利用防守动作的原始稳定风险、单脚支撑比例、关节负载与支撑裕度，定义防守基准平衡代价

$$\begin{aligned} B_d = \mathrm{clip}\Big(&0.42 r_d + 0.18\rho_{ss,d} + 0.12 u_{\tau,d} + 0.10\phi_{90,d} \\ &+ 0.08\phi_{25,d} + 0.10\delta\left(m_d^{\mathrm{mode}}\right) - 0.22 R_d^{\mathrm{bal}} \\ &- 0.10\frac{m_d^{\mathrm{sup}}}{b_{\mathrm{foot}}}, 0.02, 1.10\Big), \end{aligned} \tag{24}$$

其中 R_d^{bal} 为防守动作的平衡储备，m_d^{sup} 为残余支撑裕度。

表 4 问题二防守动作集合划分

防守组别	代表动作	主要作用机理
格挡封线类	十字格挡、单手拍挡、肘挡、下压格挡、钳制格挡	以上肢快速封闭主要来袭通道，优先削减拳法和低位腿法的直接命中。
闪避脱线类	左右侧闪、下潜闪避、后撤步、转身闪避、滑步环绕	通过横移、后撤或转体离开攻击火线，降低高速腿法与旋转攻击的直接接触概率。
姿态压缩类	护头防御、低重心防御、侧身防御	通过缩小迎击面积和下沉重心，提高多段攻击下的稳定防守能力。
平衡缓冲类	重心补偿、卸力缓冲、步点调整	通过主动退让、补步和质心回拉吸收扰动，适合冲撞和大扰动来袭。
地面过渡类	受控倒地、快速起身、倒地防御	在失稳或地面态下保护核心部位，并尽量降低二次伤害。
复合反制类	闪挡反击、下潜侧闪反击、格挡撤绕	同时利用脱线、接触控制和反击窗口，但动作更重、时序要求更高。

设攻击动作为 a，其最优攻击部位记为 $s^\star(a)$，攻击轨迹记为 $\tau(a)$，命中次数记为 h_a。基于问题一结果定义来袭时序紧迫度

$$U_a = \mathrm{clip}\left(0.40\widehat{v}_a + 0.20\widehat{\lambda}_a + 0.22\omega_{\tau(a)} + 0.18\widehat{S}_a, 0.08, 1.00\right), \tag{25}$$

再分别构造封线能力、闪避能力、机械缓冲能力、支撑稳定能力和时序匹配能力：

$$C_{a,d}^{\mathrm{blk}} = 0.56c_{d,s^\star(a)} + 0.44\alpha_{d,\tau(a)}, \tag{26}$$

$$C_{a,d}^{\mathrm{eva}} = 0.42e_{d,s^\star(a)} + 0.58\beta_{d,\tau(a)}, \tag{27}$$

$$M_d = 0.72\mu_d + 0.14\kappa_d + 0.14R_d^{\mathrm{bal}}, \tag{28}$$

$$G_d = \mathrm{clip}\left(0.42\sigma_d + 0.22R_d^{\mathrm{bal}} + 0.18\chi_d + 0.10\frac{m_d^{\mathrm{sup}}}{b_{\mathrm{foot}}} + 0.08(1-\rho_{ss,d}), 0, 0.98\right), \tag{29}$$

$$T_{a,d} = \mathrm{clip}\left(1.06 - \frac{U_a\left(0.60t_d + 0.40t_d^{\mathrm{tr}}\right)}{1.12\,t_a/h_a}, 0.02, 0.98\right). \tag{30}$$

在此基础上，构造修正后的攻防总损失函数

$$L(a,d) = 0.44D_{a,d}^{\mathrm{res}} + 0.20C_{a,d}^{\mathrm{bal}} + 0.12\widehat{E}_d + 0.14C_{a,d}^{\mathrm{tim}} + 0.12C_{a,d}^{\mathrm{seq}} - 0.08B_{a,d}^{\mathrm{ctr}}, \tag{31}$$

其中 $D_{a,d}^{\mathrm{res}}$ 表示剩余伤害，$C_{a,d}^{\mathrm{bal}}$ 表示平衡负担，$C_{a,d}^{\mathrm{tim}}$ 与 $C_{a,d}^{\mathrm{seq}}$ 分别表示时序惩罚与连续攻击惩罚，$B_{a,d}^{\mathrm{ctr}}$ 表示反击窗口收益。于是最优防守动作可写为

$$d^{\star}(a) = \arg\min_{d \in \mathcal{D}} L(a, d), \tag{32}$$

并定义近优防守集

$$\mathcal{D}^{\mathrm{near}}(a) = \left\{ d \in \mathcal{D} \mid L(a, d) \leq L\big(a, d^{\star}(a)\big) + 0.06 \right\}. \tag{33}$$

5.2.3 求解方法

本文对 13 类攻击动作与 22 类防守动作逐一配对，计算全部 13×22 组组合损失，并针对每一类攻击动作选择使 $L(a, d)$ 最小的防守动作作为最优解。同时，为刻画防守映射的替代空间，统计每种攻击动作下近优防守集的大小；为检验结论对权重设置的敏感性，对式 (31) 式中各权重进行 $\pm 12\%$ 随机扰动，重复计算 1200 次，统计最优防守保持第一名以及进入前三名的频率。

5.2.4 结果解析

问题二的最优防守匹配结果如表5 所示。

表 5　13 类攻击动作的最优防守匹配结果

攻击动作	最优防守	类型	最小损失	次优防守	第三防守	近优数
左/右直拳	单手拍挡	格挡类	0.0527	肘挡	护头防御	9
左/右勾拳	肘挡	格挡类	0.0453	单手拍挡	护头防御	11
摆拳	肘挡	格挡类	0.0350	护头防御	单手拍挡	13
组合拳	护头防御	姿态类	0.0733	肘挡	单手拍挡	10
冲撞	卸力缓冲	平衡类	0.1003	重心补偿	肘挡	2
前踢	侧身防御	姿态类	0.0644	单手拍挡	肘挡	8
膝撞	肘挡	格挡类	0.0774	护头防御	十字格挡	14
侧踢	侧身防御	姿态类	0.0619	十字格挡	肘挡	9
倒地反击	倒地防御	地面类	0.1171	护头防御	肘挡	11
低扫腿	下压格挡	格挡类	0.0242	倒地防御	低重心防御	5
拳腿组合	护头防御	姿态类	0.0865	侧身防御	肘挡	8
回旋踢/转身踢	左右侧闪	闪避类	0.1269	护头防御	转身闪避	9
五连踢	护头防御	姿态类	0.1747	肘挡	十字格挡	5

从结果可以看出，最优防守并不追求“全部通过位移躲开”，而更多依赖双脚支撑条件下的上肢封线、躯干缩姿和局部缓冲。直拳选择单手拍挡，是因为线性拳线集中、偏转效率高；勾拳、摆拳与膝撞选择肘挡，是因为近身弧线冲击更适合上臂收紧保护；前踢与侧踢选择侧身防御，是因为侧转躯干可以显著减小迎击面积；回旋踢选择左右侧

闪，说明旋转攻击更适合采用脱线思路；冲撞选择卸力缓冲，则体现出大扰动攻击更适合以退让吸能方式化解。

为了直观展示最优防守姿态，本文构造了 13 类来袭动作对应最优防守的三维动作示意，总览如图4 所示。图中黄色轨迹线表示主要防守部件的运动轨迹，其中上肢格挡与护头类动作对应手腕或肘部轨迹，侧闪与缓冲类动作对应骨盆轨迹，倒地防御对应躯干中心下沉轨迹。

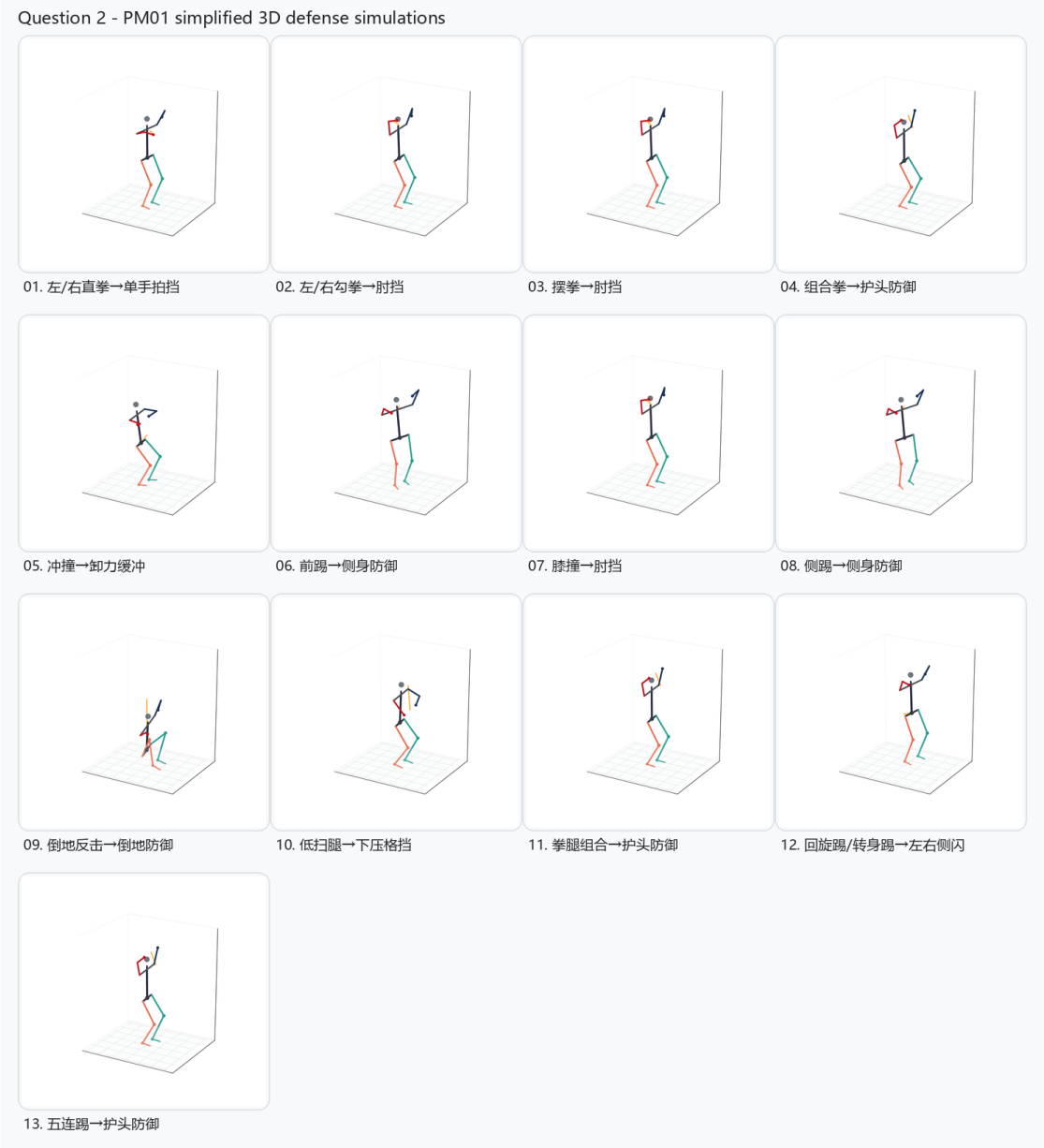

图 4　13 类来袭动作对应的最优防守动作示意

进一步绘制全部攻防组合的损失热图，如图5 所示。热图显示，拳法攻击在单手拍挡、肘挡和护头防御附近形成明显低损失带；低扫腿在下压格挡与低重心防御附近出现最低损失带；冲撞在卸力缓冲和重心补偿附近形成独立低损失区；回旋踢则在左右侧闪和转身闪避附近更优。这些低损失区与动作机理相一致，说明模型具有较好的可解释性。

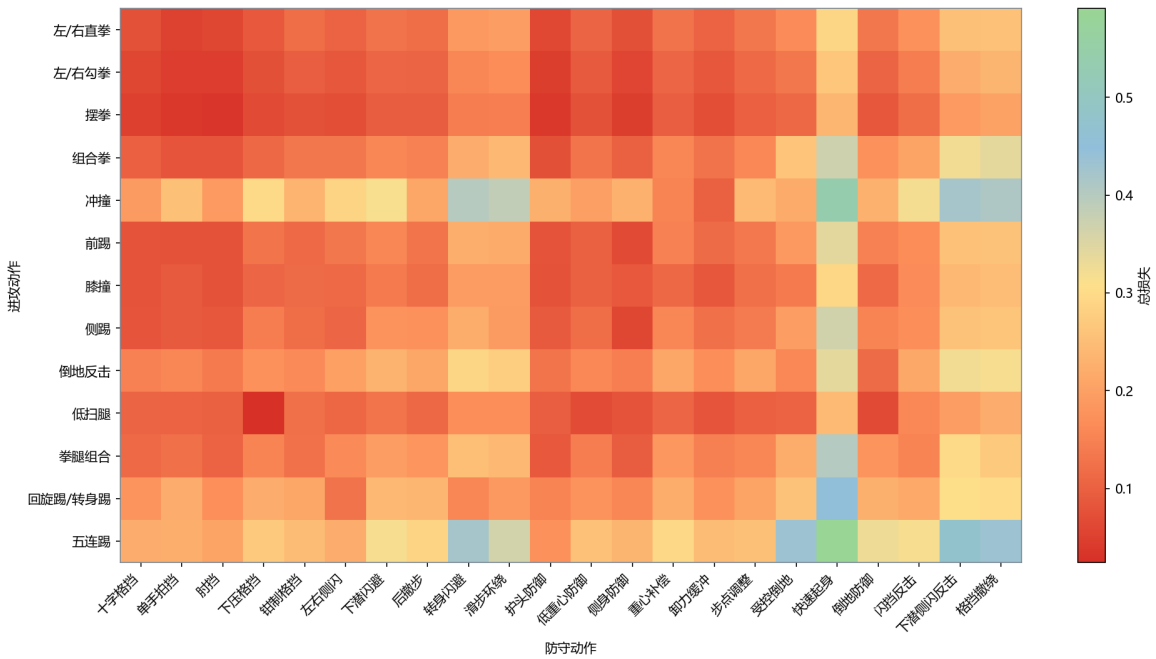

图 5 问题二攻防组合损失热图

对式 (31) 式权重进行扰动后的稳健性结果如图6 所示。结果表明，除左/右勾拳对应

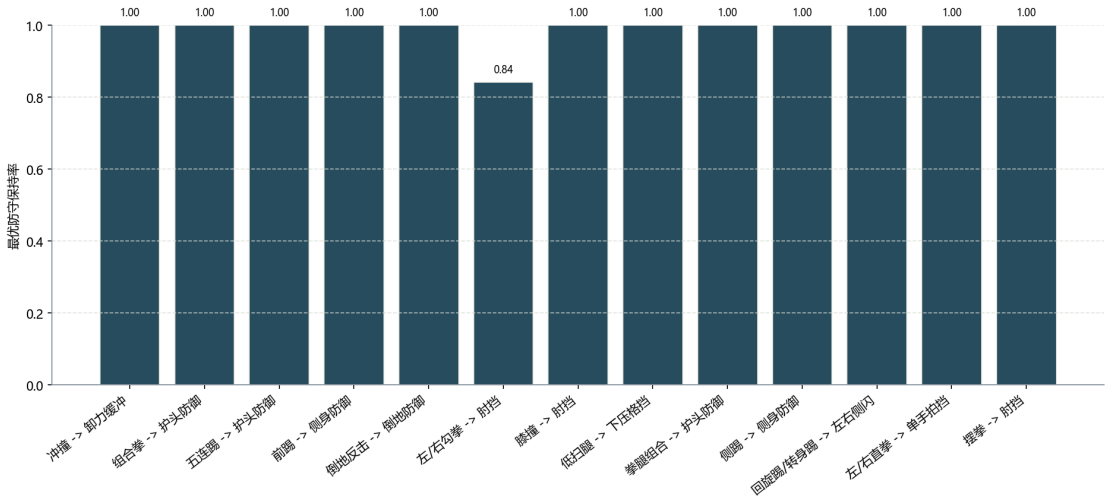

图 6 问题二最优防守在权重扰动下的稳定性

的肘挡在小幅权重扰动下会与单手拍挡、护头防御形成接近竞争外，其余攻击动作对应的最优防守保持率均达到 1.0，且全部最优防守动作进入前三名的频率均为 1.0。这说明问题二得到的攻防映射具有较强稳健性。

5.3 问题三：单场竞技赛策略优化

5.3.1 问题引入

问题三要求在忽略机器人故障的条件下，为 PM01 机器人建立单场比赛中的动作策略优化模型，并给出攻防动作的合理执行顺序。与前两问不同，问题三不再只评价单个攻击动作或单次攻防匹配，而是要处理连续对抗中的状态演化问题：当前动作可能带来即时得分，也可能使自身平衡风险上升；当前防守虽然减少输出，但可以恢复姿态、拉开距离并为后续反击创造条件。因此，问题三的核心是建立一个随机体完好度状态、平衡状态与距离状态变化的动态策略模型。

5.3.2 模型构建

设第 t 个战术步的比赛状态为

$$s_t = \left(h_t^{\text{self}}, h_t^{\text{opp}}, b_t^{\text{self}}, b_t^{\text{opp}}, d_t\right), \tag{34}$$

其中机体完好度取离散集合 $\mathcal{H} = \{0, 0.25, 0.50, 0.75, 1.00\}$，平衡风险取离散集合 $\mathcal{B} = \{0, 0.33, 0.66, 1.00\}$，距离状态取 $\mathcal{D}_r = \{\text{近距}, \text{中距}, \text{远距}\}$，从而形成 $5\times5\times4\times4\times3 = 1200$ 个状态。去除终止状态后，参与策略优化的非终止状态共有 432 个。

动作集合由问题一的 13 类攻击动作、问题二的 22 类防守动作及一个原地调整动作组成，即

$$\mathcal{U} = \mathcal{A} \cup \mathcal{D} \cup \{u_0\}, \qquad |\mathcal{U}| = 36, \tag{35}$$

并根据距离匹配和当前平衡风险定义状态相关的可行动作集 $\mathcal{U}(s_t)$。

若我方在状态 s_t 下选择动作 u_t，对手选择动作 v_t，则我方对对手造成的有效伤害写为

$$G_t^{\text{out}} = D_{u_t}\rho(u_t, d_t)m(u_t, v_t)\left(0.86 + 0.14\alpha_{u_t}\right), \tag{36}$$

其中 $m(u_t, v_t)$ 由问题二的拦截得分与总损失折算得到。机体完好度随伤害累积而下降，平衡风险同时受到自身动作风险、对手打击扰动和防守动作恢复能力影响，对我方有

$$b_{t+1}^{\text{self}} = \Pi_{\mathcal{B}}\left[\text{clip}\left(b_t^{\text{self}} + r(u_t) + 0.42G_t^{\text{in}} - c(u_t) - \delta(u_t), 0, 1\right)\right], \tag{37}$$

距离状态则由双方动作的趋近、后撤和碰撞意图共同决定。

为了统一刻画得分收益与姿态安全性，定义单步收益函数

$$R_t(s_t, u_t, v_t) = 1.18\left(G_t^{\text{out}} - G_t^{\text{in}}\right) + 0.76\left[b_{t+1}^{\text{opp}} - b_t^{\text{opp}}\right]^+ \tag{38}$$

$$- 1.05\left[b_{t+1}^{\text{self}} - b_t^{\text{self}}\right]^+ - C_E(u_t) + B_{\text{term}} + B_{\text{ini}} + B_{\text{pre}} + B_{\text{def}} - P_{\text{idle}}, \tag{39}$$

其中

$$C_E(u_t) = 0.10E_{u_t} + 0.025t_{u_t}. \tag{40}$$

终止价值定义为

$$V_T(s) = 2.6\left(h^{\text{self}} - h^{\text{opp}}\right) + 1.35\left(b^{\text{opp}} - b^{\text{self}}\right) + 1.8I_{\text{win}} - 1.8I_{\text{loss}}. \tag{41}$$

5.3.3 求解方法

由于对手不是静止环境，而会根据我方动作进行反制，本文采用有限时域鲁棒马尔可夫博弈模型求解。对任意状态 s、我方动作 u 与对手动作 v，先定义

$$Q_t(s,u,v) = R_t(s,u,v) + \gamma V_{t+1}\left(f(s,u,v)\right), \tag{42}$$

其中 $f(s,u,v)$ 为状态转移函数，$\gamma = 0.955$ 为折扣因子。进一步采用“最坏响应 + 平均响应”的混合鲁棒准则建立 Bellman 递推：

$$V_t(s) = \max_{u\in\mathcal{U}(s)}\left[\beta \min_{v\in\mathcal{U}_{\mathrm{opp}}(s)} Q_t(s,u,v) + (1-\beta)\overline{Q}_t(s,u) + \eta_t I_{\mathrm{atk}}(u) + 0.16 I_{\mathrm{def}}(u) I_{b^{\mathrm{self}}\geq 0.66}\right], \tag{43}$$

其中 $\beta = 0.72$ 为鲁棒系数。自终止层向前递推即可得到每一战术步、每一状态下的最优动作。计算表明，在初始均势状态

$$s_0 = (1.00, 1.00, 0, 0, \text{中距})$$

下的价值为 1.8403，在 432 个非终止状态中，有 368 个状态的初始价值为正，占比 85.19%，说明在给定动作库和状态转移规则下，PM01 可以通过动态切换进攻与防守获得较高的胜势倾向。

5.3.4 结果解析

对全部非终止状态统计开局时刻的最优动作分布，结果如表6 与图7 所示。 结果表

表 6 问题三开局时刻非终止状态的最优动作分布

动作	类型	状态数	占比
左/右直拳	进攻	133	30.79%
卸力缓冲	防守	87	20.14%
护头防御	防守	80	18.52%
冲撞	进攻	68	15.74%
侧身防御	防守	43	9.95%
受控倒地	防守	18	4.17%
步点调整	防守	3	0.69%

明，问题三并没有给出“全程强攻”的策略。左/右直拳占比最高，是因为其动作幅度小、恢复快、距离适应性好，能够在近距和中距稳定输出；卸力缓冲与护头防御合计占比达到 38.66%，说明当自身平衡风险、对手攻击威胁或距离不利时，模型更倾向于先稳住姿态；冲撞则主要出现在远距或中距稳定状态，用于快速压缩距离并制造对手平衡扰动。

进一步抽取典型状态下的策略热图，如图8 所示。结合策略热图和状态分类统计，可将问题三的单场比赛方案整理为表7。因此，问题三的推荐策略不是一条固定动作序

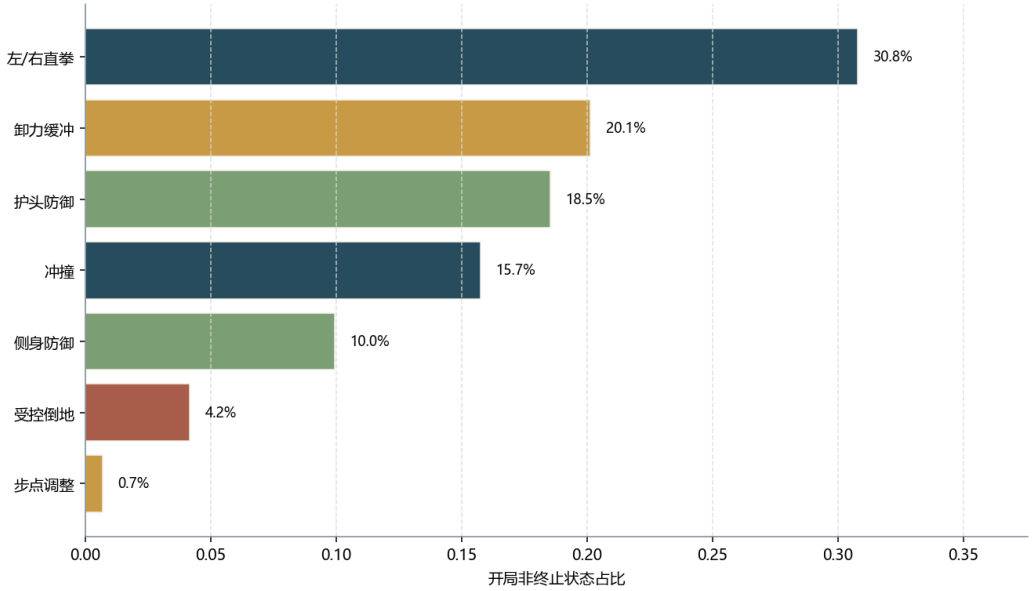

图 7 问题三开局策略动作分布

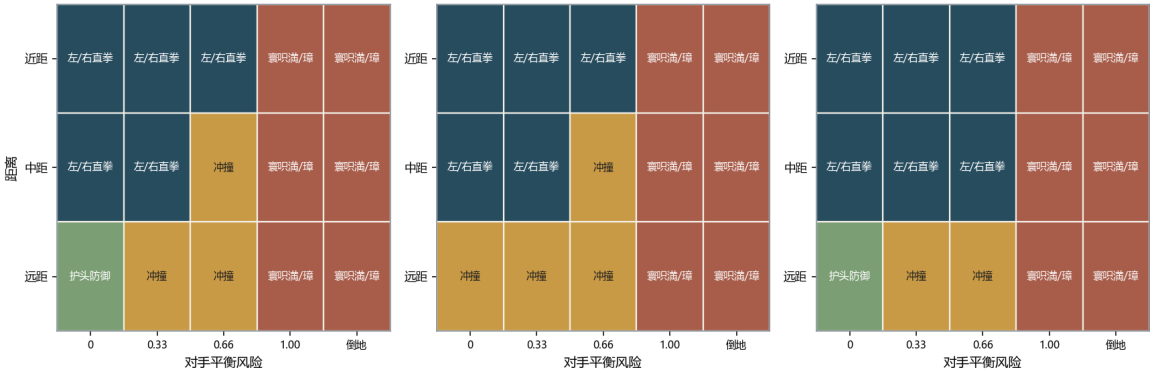

图 8 问题三典型状态最优动作热图

列，而是一套状态反馈策略。若比赛初始为双方机体完好度均处于满值、低平衡风险、中距对峙，则可采用“直拳试探，必要时以冲撞重新压近，近距后继续用拳法得分”的开局思路；当自身平衡风险上升到 0.66 附近时，应立刻转入护头、防侧身或卸力缓冲等稳定化动作，待重心恢复后再重新进入进攻节奏。

为检验策略的抗反制能力，本文从初始均势状态出发，令对手每一步都选择使我方后续价值最小的响应动作，得到一条最坏响应压力测试轨迹，如图9 所示。压力测试共形成 14 个有效动作步，总动作时间约为 31.342 s。在最坏响应下，我方累计输出伤害为 1.0529，累计承受伤害为 2.5381，最终压力测试价值降至 −0.65。这说明当对手连续采用强反制时，模型会从积极压制自动切换到侧身防御等稳定化动作，表明平衡风险项确实触发了防守恢复机制。综合来看，问题三得到的关键结论是：PM01 的单场最优策略应采用“稳定状态主动压制，高风险状态立即恢复”的阈值切换结构。

表 7 PM01 单场比赛攻防策略方案

阶段	触发条件	推荐动作	策略含义
A 建势试探	中距或远距，且 $b_t^{\text{self}} < 0.33$	冲撞、左/右直拳，必要时接护头防御	利用冲撞压缩距离，用直拳进行低风险试探，避免一开始使用高幅度腿法
B 近距得分	近距，且 $b_t^{\text{self}} < 0.66$	左/右直拳为主，勾拳和摆拳作为变化	在近身状态下用恢复快的拳法累积收益，不把高平衡风险动作作为连续主攻
C 中距压制	中距稳定，且对手 $b_t^{\text{opp}} \geq 0.33$	左/右直拳、冲撞、短组合进攻	对手平衡已受扰动时增加主动性，以直线动作继续压迫而非盲目旋转
D 失稳恢复	$b_t^{\text{self}} \geq 0.66$	护头防御、卸力缓冲、侧身防御、步点调整	优先降低受击面积和姿态扰动，先恢复双脚支撑和重心位置
E 高危兜底	机体完好度落后且平衡风险接近 1.00	受控倒地、倒地防御、快速起身	接受短时低姿态，换取多接触支撑，避免被动摔倒导致终止损失

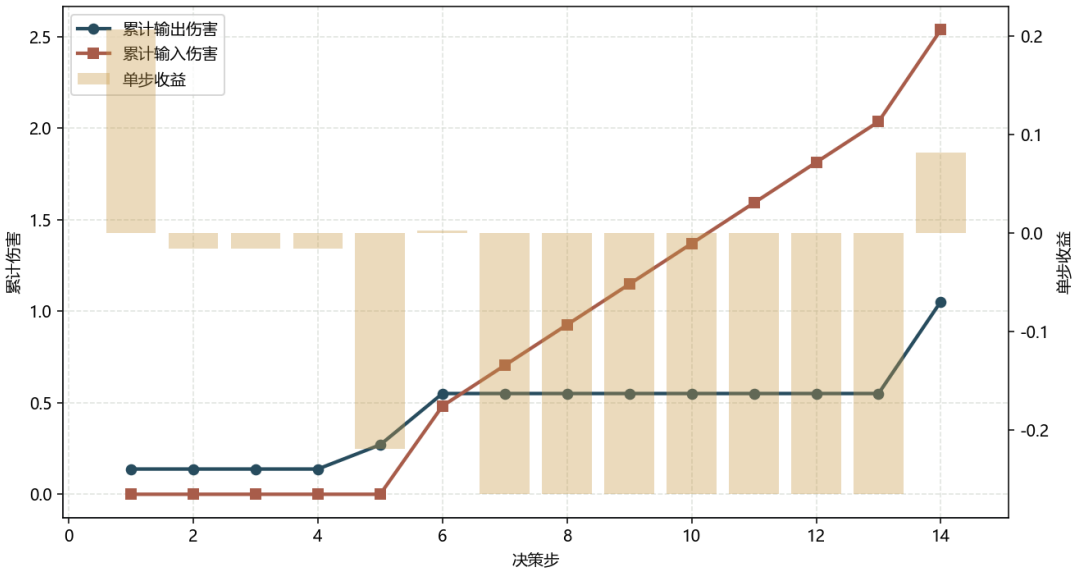

图 9 问题三初始状态最坏响应压力测试轨迹

5.4 问题四：BO3 赛制下资源使用决策优化

5.4.1 问题引入

问题四在问题三单场攻防策略的基础上进一步引入故障风险和资源约束。PM01 在作战过程中可能出现轻微故障、中等故障乃至严重故障，而赛事规则又限制人工复位最多 2 次、战术暂停最多 2 次、紧急故障维修最多 1 次。若资源使用过早，当前局胜率可能提高，但后续局缺少补救手段；若资源使用过晚，则可能在本局中因故障或失稳直接丧失优势。因此，第四问需要从单局最优转向 BO3 三局两胜赛制下的全局最优。

5.4.2 模型构建

定义 BO3 决策状态为

$$x_t = (w_t, l_t, k_t, a_t, \phi_t, e_t, b_t, p_t, r_t, m_t)\,, \tag{44}$$

其中 w_t, l_t 表示系列赛比分，k_t 表示当前局内的高层资源决策步，a_t 表示局内优势，ϕ_t 表示故障等级，e_t 表示能量等级，b_t 表示平衡风险，p_t, r_t, m_t 分别表示战术暂停、人工复位和紧急维修的剩余次数。资源约束为

$$p_t \in \{0,1,2\}, \quad r_t \in \{0,1,2\}, \quad m_t \in \{0,1\}, \tag{45}$$

可选资源动作集合为

$$\Omega(x_t) = \{\text{fight}, \text{pause}, \text{reset}, \text{repair}\}. \tag{46}$$

第四问的局内常规攻防能力不重新主观设定，而由前三问结果汇总得到。设问题一前 5 个攻击动作的归一化均值为 A_q，问题二最优防守损失与近优防守数量折算得到防守鲁棒性 D_q，问题三初始状态价值、胜势状态比例和最坏响应压力测试价值分别为 $V_0^{(3)}$、$\rho_w^{(3)}$ 和 $V_{\text{press}}^{(3)}$。据此定义战术基础偏置

$$\begin{aligned}\chi = {} & 0.060\text{clip}\left(\frac{V_0^{(3)}}{2.2}, -1, 1\right) + 0.130\left(\rho_w^{(3)} - 0.5\right) + 0.018V_{\text{press}}^{(3)} \\ & + 0.035(A_q - 0.75) + 0.030(D_q - 0.75).\end{aligned} \tag{47}$$

在本文数据中，$A_q = 0.7980$，$D_q = 0.9548$，$V_0^{(3)} = 1.8403$，$\rho_w^{(3)} = 0.8519$，$V_{\text{press}}^{(3)} = -0.6500$，由式 (47) 式得 $\chi = 0.0653$。

当选择常规攻防 fight 时，局面优势按 $\Delta a_t \in \{-1, 0, 1\}$ 随机变化。设综合攻防边际为

$$E(x_t) = \chi + \xi_c + 0.120a_t + 0.065(e_t - 1) - \lambda_\phi(\phi_t) - \lambda_b(b_t), \tag{48}$$

则局面扩大与恶化概率分别写为

$$\begin{aligned}\Pr(\Delta a_t = 1) = {} & \text{clip}\,[0.285 + 0.170\tanh(1.55E) - 0.020\phi_t \\ & -0.015b_t + 0.014(e_t - 1), 0.045, 0.600]\,,\end{aligned} \tag{49}$$

$$\begin{aligned}\Pr(\Delta a_t = -1) = {} & \text{clip}\,[0.270 - 0.165\tanh(1.55E) + 0.032\phi_t \\ & +0.022b_t - 0.010(e_t - 1), 0.045, 0.650]\,.\end{aligned} \tag{50}$$

故障发生或恶化概率定义为

$$P_\phi(x_t) = \mathrm{clip}\Big[0.050 + 0.018\left\lfloor \frac{k_t}{4} \right\rfloor + \zeta_c + 0.034b_t + 0.030(1-e_t)^+ + 0.040\phi_t + 0.020I_{a_t<0}, 0.015, 0.430\Big], \tag{51}$$

若故障和平衡风险同时较高，还存在本局提前失利概率

$$P_{\mathrm{break}}(x_t) = \mathrm{clip}\Big[0.030I_{\phi_t\geq 2\vee b_t\geq 2} + 0.055\phi_t + 0.050b_t + 0.080I_{\phi_t=3} + 0.070I_{b_t=3} - 0.030e_t, 0, 0.520\Big]. \tag{52}$$

若选择资源动作，则状态按资源机理更新：战术暂停主要恢复能量并降低平衡风险，人工复位主要消除姿态临界，紧急维修主要清除故障但占用更多决策窗口。

5.4.3 求解方法

设 $F(x)$ 为状态 x 下最终赢得 BO3 的最大概率。若 $w = 2$，则 $F(x) = 1$；若 $l = 2$，则 $F(x) = 0$。当当前局已经走完 12 个决策窗口时，先根据局内末态计算本局胜率

$$\pi(x) = \sigma\left[\chi + \xi_c^{\mathrm{end}} + 0.950a_t + 0.160(e_t - 1) - 0.360b_t - 0.470\phi_t\right], \tag{53}$$

并利用

$$F(x) = \pi(x)F(x^{\mathrm{win}}) + [1-\pi(x)]\,F(x^{\mathrm{lose}}) \tag{54}$$

进行局间递推。若当前局尚未结束，则采用

$$F(x) = \max_{u\in\Omega(x)} \sum_{x'} P(x'|x,u)F(x') \tag{55}$$

进行逆向动态规划，即可求得每个 BO3 状态下应选择常规攻防、暂停、复位还是维修。

5.4.4 结果解析

在初始满资源状态

$$(w, l, k, a, \phi, e, b, p, r, m) = (0, 0, 0, 0, 0, 2, 0, 2, 2, 1)$$

下，模型给出的最优首步为常规攻防，不建议局前预支资源，最终 BO3 获胜概率为 0.4341。这说明在显式考虑故障恶化和提前失利概率后，资源更应该保留给中后局的关键状态，而不是在局前提前消耗。

满资源关键局分状态的决策结果见表8。

进一步统计不同局分场景下的典型最优资源动作，得到图10。图中横轴为故障等级，纵轴为局内局面，颜色和文字表示该状态下最优资源动作。结果显示，战术暂停主要出现在无故障或轻微预警但局面不够稳的状态，尤其在落后局和决胜局中更常见；紧急维

表 8 问题四满资源关键状态决策结果

状态	场景	最优动作	系列赛获胜概率	解释
(0, 0)	开局平局	常规攻防	0.4341	首局不预支资源，保留 BO3 弹性
(1, 0)	领先	常规攻防	0.7748	已握赛点，稳定推进优于局前消耗
(0, 1)	落后	常规攻防	0.2777	开局仍先争取常规得分，资源留给高危状态
(1, 1)	决胜局	常规攻防	0.6161	无故障且能量充足时不立即暂停

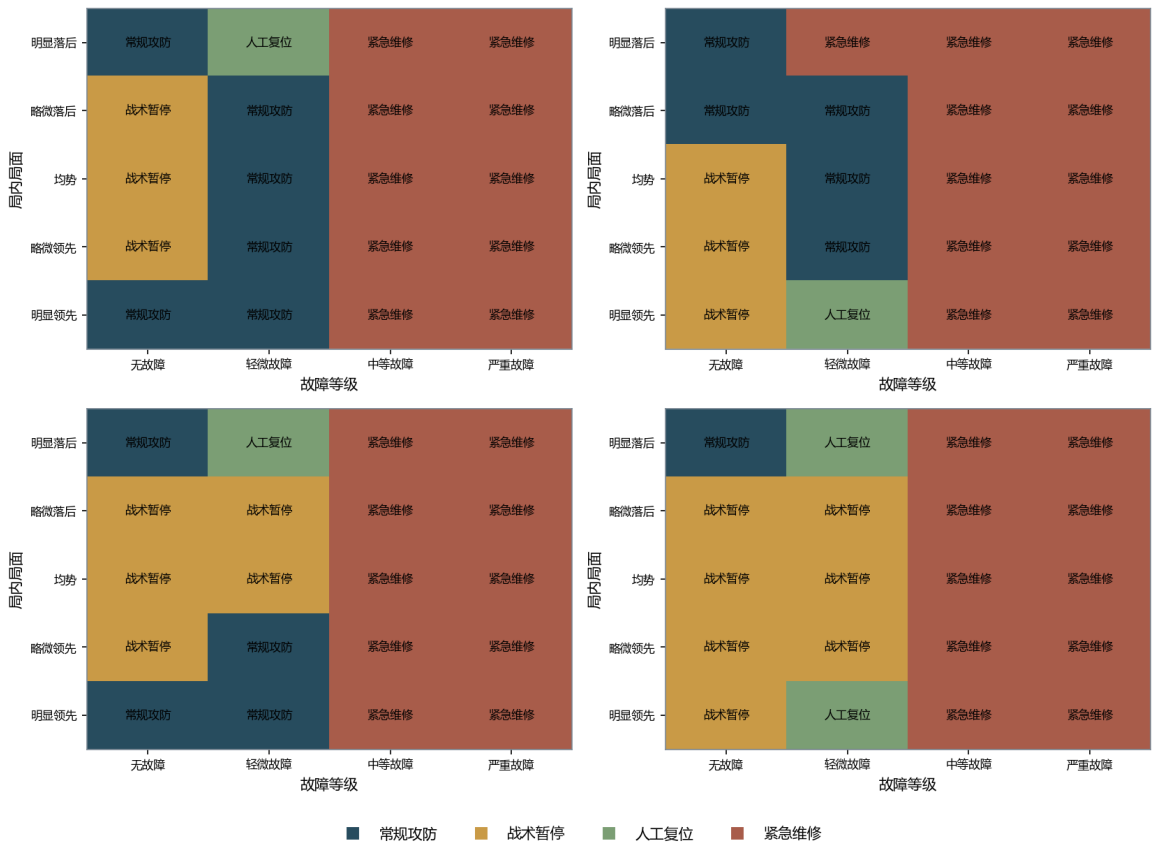

图 10 问题四不同局分场景下的最优资源动作

修主要集中于中等故障和严重故障状态；人工复位则主要出现在高平衡风险但维修时间代价过大的状态。在一个代表性决胜高压状态

$$(1, 1, k = 9, a = -1, \phi = 2, e = 0, b = 2, p = 1, r = 1, m = 1)$$

下，继续常规攻防的系列赛价值为 0.1473，战术暂停为 0.2187，人工复位为 0.2369，紧急维修为 0.2218，因此最优选择为人工复位。这说明当距离本局结束已不远时，复位比维修更能快速消除倒地风险。

资源边际价值如表9 和图11 所示。结果表明，战术暂停的边际价值最高，满资源相

表 9 问题四初始状态资源边际价值

资源维度	低资源价值	高资源价值	价值提升
战术暂停次数	0.2430	0.4341	0.1911
人工复位次数	0.3106	0.4341	0.1236
紧急维修次数	0.3425	0.4341	0.0916

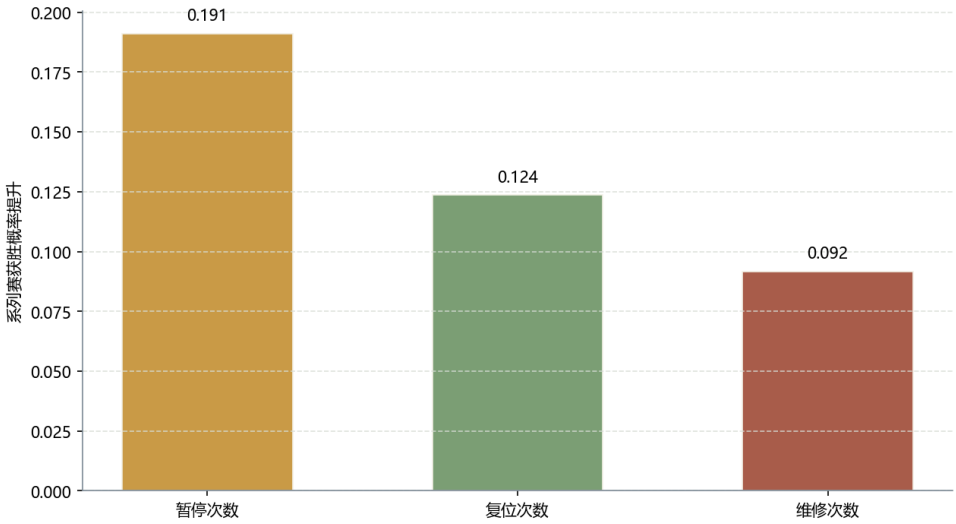

图 11 问题四资源边际价值

对无暂停时可使系列赛获胜概率提高 0.1911；人工复位次之，提高 0.1236；紧急维修的全局边际提升为 0.0916。这并不表示维修不重要，而是说明维修触发场景更集中：只有进入中高等级故障时，维修才显著优于其他动作；在无故障或轻微故障状态下，保留维修比提前使用更优。

为了检验资源策略是否满足“该用时用、不该用时保留”的要求，本文从四个典型局分状态出发，按照动态规划策略进行前向概率传播，统计各资源的期望使用次数，结果如图12 所示。在开局平局场景中，最优策略下暂停、复位和维修的期望使用次数分别为 1.7890、1.4421 和 0.6735；在领先场景中分别为 1.8173、0.8208 和 0.4207；在落后场景中分别为 1.5845、1.0799 和 0.5012；在决胜局中分别为 1.9805、0.7059 和 0.1653。从

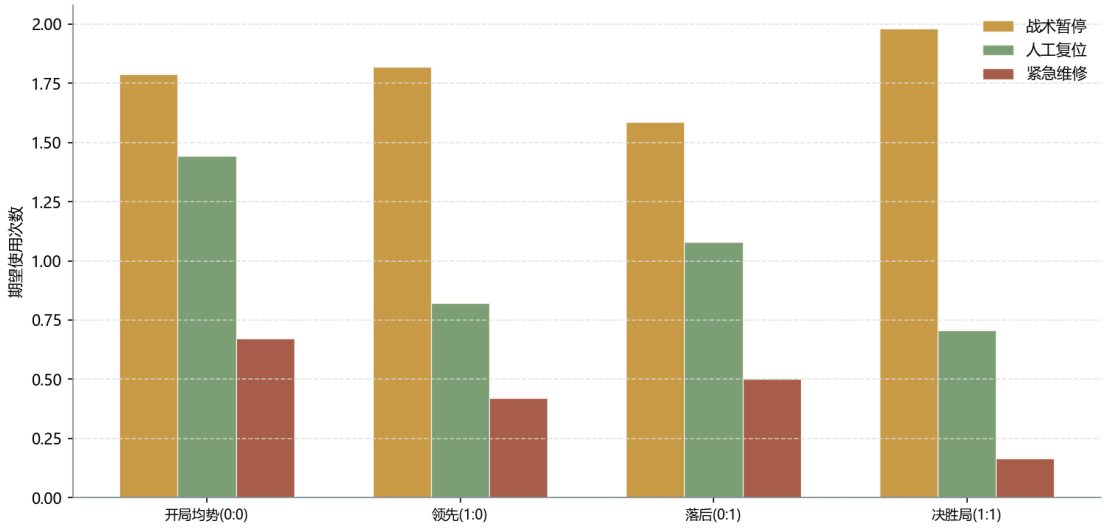

图 12 问题四典型局分场景下的资源使用强度

首次触发时间看，各资源多集中在 140 ~ 190 s 的中后局窗口，说明模型并不鼓励开局立即透支资源，而是把资源留给局面分化和故障风险开始显著累积的阶段。

将上述结果整理为可执行的 BO3 作战资源方案，如表10 所示。综上，第四问得到

表 10 问题四 BO3 资源使用决策方案

场景	资源使用时机	建议次数倾向	执行目标
开局平局 (0, 0)	无故障满能量时不预支；中后局能量下降或轻微落后时优先暂停；中高故障时维修	暂停约 1–2 次，复位约 1 次以上，维修视故障触发	保留后续局资源弹性，同时避免首局故障滚雪球
领先 (1, 0)	稳定状态继续常规攻防；轻微风险用暂停吸收；非严重故障少用维修	暂停约 1–2 次，复位和维修偏保守	以守住赛点为主，不因过度维修浪费终结机会
落后 (0, 1)	轻微落后或能量不足时更积极暂停；倒地临界时复位；中高故障及时维修	暂停约 1–2 次，复位约 1 次，维修按故障触发	当前局必须争胜，资源保留价值下降
决胜 (1, 1)	资源不再跨局保留；无故障仍常规攻防；中等故障与高平衡风险优先复位或维修	暂停接近用满，复位用于高危，维修用于严重故障	最大化本局胜率，避免资源剩余而输掉比赛

的核心结论为：第一，满资源且无故障时不应局前预支资源；第二，战术暂停是全局边际价值最高的资源，应作为能量下降、轻微落后和平衡预警时的优先干预手段；第三，

人工复位适合倒地临界、低能量且时间不足以维修的状态；第四，紧急维修应保留给中等及严重故障，尤其是首局和落后局中可能影响后续 BO3 走势的故障状态。

5.5 问题五：面向具身智能产业发展的建议

5.5.1 建议形成依据

问题五要求在前四问建模结果基础上，面向机器人未来应用场景提出促进我国具身智能产业发展的合理建议。前四问已经形成了较完整的证据链：问题一说明，高价值动作往往不是极限威力最大的动作，而是稳定性、能耗和战术可用性综合较优的动作；问题二说明，防守策略可以整理成可复现的攻防映射模板；问题三说明，机器人比赛策略应根据平衡风险和距离状态动态切换，而不能依赖固定连招；问题四说明，暂停、复位和维修等资源的使用时机会显著影响 BO3 最终胜率。由此可见，具身智能产业发展不能只关注单个模型或单个硬件指标，而应同时建设动作标准、安全闭环、仿真训练、赛中辅助和场景验证体系。

为避免建议流于口号，本文从产业影响度 M、风险紧迫度 R、实施可行性 G 和协同带动性 C 四个方面对建议进行优先级评估，定义

$$P_i = 0.30M_i + 0.25R_i + 0.25G_i + 0.20C_i, \tag{56}$$

其中 P_i 越大，表示该建议越应优先推进。根据前四问模型结果，得到表11。

表 11 问题五建议优先级评估

建议方向	影响度	紧迫度	可行性	协同性	优先级
统一动作-状态-故障数据标准	0.94	0.88	0.86	0.95	0.9115
建设平衡-故障-资源安全基线	0.96	0.93	0.82	0.92	0.9145
构建攻防策略仿真训练闭环	0.90	0.84	0.78	0.90	0.8520
部署赛中资源决策辅助系统	0.86	0.82	0.84	0.87	0.8475
推进分级场景化应用试点	0.88	0.80	0.80	0.86	0.8370

5.5.2 建议书正文

基于前四问的建模结果，本文提出如下建议。

第一，建立统一的动作、状态与故障数据标准。建议由赛事组织方联合机器人企业和高校团队，统一动作名称、关节轨迹、支撑状态、平衡风险、故障等级、能耗和资源使用记录等字段，形成可共享的数据接口。问题一和问题二表明，动作特征表征与攻防映射只有在统一口径下才能复现和比较，因此标准建设应先于大规模算法竞赛和应用推广。

第二，建设平衡—故障—资源联动的安全基线。问题三显示，当平衡风险升高时，机器人必须从主动进攻切换到防守恢复；问题四进一步表明，暂停、复位和维修等资源

的使用时机会显著影响 BO3 胜率。建议将平衡预警、故障识别、资源触发和恢复动作固化为底层安全模块，形成“监测—预警—干预—恢复”的闭环，避免机器人在高动态任务中因单次失稳或小故障引发连锁失败。

第三，构建攻防策略仿真训练闭环。建议以赛事数据为牵引，建设“数字孪生仿真—动作策略评估—真实机器人验证—参数回灌”的训练平台。问题二的防守映射和问题三的状态反馈策略可以直接转化为仿真对抗样本，用于训练机器人在不同距离、平衡风险和故障状态下选择可解释动作，而不是只追求黑箱端到端策略。

第四，部署赛中资源决策辅助系统。问题四结果显示，战术暂停的全局边际价值最高，人工复位和紧急维修也具有明确的触发场景。建议将资源动态规划结果封装为教练辅助工具，在比赛和训练中实时提示“是否暂停、是否复位、是否维修”，重点提供可解释的概率变化和风险说明。

第五，推进分级场景化应用试点。竞技策略中的动作稳定性、故障恢复和资源调度思想，可迁移到巡检、应急救援、危险作业、康养陪护、教育训练等场景。建议先在封闭或半封闭场景中开展试点，逐步验证安全边界、任务收益和维护成本，再向开放环境推广。场景推进应遵循“先安全、后性能；先标准、后规模；先可验证、后商业化”的节奏。

综合来看，具身智能产业发展的关键不是单纯追求某一个高难动作或某一次比赛胜利，而是建立可复现、可解释、可恢复、可迁移的工程体系。建议近期优先完成数据标准和安全基线建设，中期形成仿真训练闭环与赛中决策辅助能力，远期推动多行业场景验证和规模化应用，使机器人竞技中的技术积累能够真正转化为我国具身智能产业的长期竞争力。

参考文献

[1] Kajita S, Hirukawa H, Harada K, Yokoi K. Introduction to Humanoid Robotics[M]. Berlin: Springer, 2014.

[2] Siciliano B, Khatib O. Springer Handbook of Robotics[M]. 2nd ed. Cham: Springer, 2016.

[3] Bertsekas D P. Dynamic Programming and Optimal Control[M]. 4th ed. Belmont: Athena Scientific, 2017.

[4] Sutton R S, Barto A G. Reinforcement Learning: An Introduction[M]. 2nd ed. Cambridge: MIT Press, 2018.

[5] LaValle S M. Planning Algorithms[M]. Cambridge: Cambridge University Press, 2006.

[6] Pratt J, Carff J, Drakunov S, Goswami A. Capture point: A step toward humanoid push recovery[C]// 6th IEEE-RAS International Conference on Humanoid Robots. Genova: IEEE, 2006: 200-207.

[7] EngineAI Robotics. engineai_ros2_workspace[EB/OL]. GitHub repository. https://github.com/engineai-robotics/engineai_ros2_workspace, 2026-04-19.

附录：核心求解代码

附录仅保留直接用于问题一至问题四求解的核心程序，删除参数提取、数据整理与三维渲染脚本。

问题一核心代码

```python
#!/usr/bin/env python3
"""Solve MathorCup B Question 1 with stability-aware PM01 action portraits."""

from __future__ import annotations

import csv
import json
import math
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Tuple

import matplotlib
import numpy as np
from matplotlib import pyplot as plt

matplotlib.use("Agg")

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False


WORKSPACE_ROOT = Path(__file__).resolve().parents[1]
PORTRAIT_JSON = WORKSPACE_ROOT / "data" / "question1_action_portraits" / "q1_action_portraits_pm01.json"
INTEGRATED_TABLE = WORKSPACE_ROOT / "data" / "pm01_repo_extract" / "pm01_integrated_table.csv"
OUTPUT_DIR = WORKSPACE_ROOT / "outputs" / "q1_complete_solution"


ACTION_HITS = {
    "straight_punch": 1,
    "hook_punch": 1,
    "combo_punch": 2,
    "swing_punch": 1,
    "front_kick": 1,
    "side_kick": 1,
    "spin_kick": 1,
    "low_sweep": 1,
    "knee_strike": 1,
    "punch_kick_combo": 2,
    "five_kick_combo": 5,
    "charge": 1,
    "ground_counter": 2,
}


TACTICAL_AVAILABILITY = {
    "straight_punch": 0.98,
    "hook_punch": 0.95,
    "combo_punch": 0.92,
    "swing_punch": 0.90,
    "front_kick": 0.84,
    "side_kick": 0.78,
    "spin_kick": 0.62,
    "low_sweep": 0.74,
    "knee_strike": 0.80,
    "punch_kick_combo": 0.76,
    "five_kick_combo": 0.48,
    "charge": 0.60,
    "ground_counter": 0.18,
}


SITE_NAME_CN = {"H": "头部", "T": "躯干", "L": "四肢"}
SITE_DAMAGE = {"H": 1.00, "T": 0.82, "L": 0.58}
SITE_EXPOSURE = {"H": 0.76, "T": 1.00, "L": 0.88}
SITE_HEIGHT = {"H": 1.30, "T": 0.92, "L": 0.46}
SUPPORT_MODE_PENALTY = {
    "double_support": 0.00,
    "transition_support": 0.08,
    "single_support": 0.12,
    "alternating_support": 0.18,
    "multi_contact_ground": -0.05,
}


@dataclass(frozen=True)
class RobotContext:
    total_mass_kg: float
```

```python
    arm_total_m: float
    leg_total_m: float
    torso_m: float
    foot_width_m: float
    heavy_effort_nm: float
    light_effort_nm: float
    v_ref_punch: float
    v_ref_kick: float
    v_ref_combo: float
    v_ref_special: float
    r_ref_punch: float
    r_ref_kick: float
    r_ref_combo: float
    r_ref_special: float


@dataclass(frozen=True)
class ActionPortrait:
    action_key: str
    action_name_cn: str
    category: str
    m_eff_ratio: float
    speed_util: float
    q25_count: int
    q90_count: int
    torque_util: float
    excursion_q25: float
    excursion_q90: float
    duration_s: float
    zmp_ratio: float
    angmom_ratio: float
    com_height_ratio: float
    asymmetry: float
    precision_h: float
    precision_t: float
    precision_l: float
    support_mode: str
    single_support_ratio: float
    com_drop_m: float
    support_margin_m: float
    torso_pitch_comp_rad: float
    torso_roll_comp_rad: float
    torso_yaw_comp_rad: float
    arm_counter_pitch_rad: float
    arm_counter_roll_rad: float
    arm_counter_yaw_rad: float
    knee_flex_ratio: float
    stability_score: float


def clamp(value: float, lower: float, upper: float) -> float:
    return max(lower, min(upper, value))


def load_summary_table() -> Dict[str, float]:
    summary: Dict[str, float] = {}
    with INTEGRATED_TABLE.open("r", encoding="utf-8-sig", newline="") as handle:
        reader = csv.DictReader(handle)
        for row in reader:
            if row["section"] != "summary":
                continue
            try:
                summary[row["field_name"]] = float(row["value"])
            except ValueError:
                continue
    return summary


def load_payload() -> Dict[str, object]:
    with PORTRAIT_JSON.open("r", encoding="utf-8") as handle:
        return json.load(handle)


def build_robot_context(payload: Dict[str, object], summary: Dict[str, float]) -> RobotContext:
    basis = payload["robot_basis"]
    arm_total = float(basis["upper_arm_length_m"]) + float(basis["forearm_length_m"])
    leg_total = float(basis["thigh_length_m"]) + float(basis["shank_length_m"])
    torso = float(basis["torso_length_m"])

    upper_velocity = (
        summary["joint_velocity_limit.J13_SHOULDER_PITCH_L"]
        + summary["joint_velocity_limit.J14_SHOULDER_ROLL_L"]
        + summary["joint_velocity_limit.J16_ELBOW_PITCH_L"]
    ) / 3.0
    lower_velocity = (
        summary["joint_velocity_limit.J00_HIP_PITCH_L"]
        + summary["joint_velocity_limit.J03_KNEE_PITCH_L"]
        + summary["joint_velocity_limit.J04_ANKLE_PITCH_L"]
    ) / 3.0
    waist_velocity = summary["joint_velocity_limit.J12_WAIST_YAW"]
```

```
    v_ref_punch = 0.45 * upper_velocity * arm_total
    v_ref_kick = 0.38 * lower_velocity * leg_total
    v_ref_combo = 0.5 * (v_ref_punch + v_ref_kick)
    v_ref_special = 0.30 * (lower_velocity * leg_total + waist_velocity * torso)

    return RobotContext(
        total_mass_kg=float(basis["total_mass_kg"]),
        arm_total_m=arm_total,
        leg_total_m=leg_total,
        torso_m=torso,
        foot_width_m=float(basis["foot_width_m"]),
        heavy_effort_nm=float(basis["heavy_effort_limit"]),
        light_effort_nm=float(basis["light_effort_limit"]),
        v_ref_punch=v_ref_punch,
        v_ref_kick=v_ref_kick,
        v_ref_combo=v_ref_combo,
        v_ref_special=v_ref_special,
        r_ref_punch=arm_total,
        r_ref_kick=0.95 * leg_total,
        r_ref_combo=0.5 * (arm_total + leg_total),
        r_ref_special=0.5 * (torso + leg_total),
    )


def load_actions(payload: Dict[str, object]) -> List[ActionPortrait]:
    actions: List[ActionPortrait] = []
    for item in payload["actions"]:
        profile = item["pm01_calibrated_profile"]
        actions.append(
            ActionPortrait(
                action_key=str(profile["action_key"]),
                action_name_cn=str(profile["action_name_cn"]),
                category=str(profile["category"]),
                m_eff_ratio=float(profile["m_eff_ratio"]),
                speed_util=float(profile["speed_util"]),
                q25_count=int(profile["q25_count"]),
                q90_count=int(profile["q90_count"]),
                torque_util=float(profile["torque_util"]),
                excursion_q25=float(profile["excursion_q25"]),
                excursion_q90=float(profile["excursion_q90"]),
                duration_s=float(profile["duration_s"]),
                zmp_ratio=float(profile["zmp_ratio"]),
                angmom_ratio=float(profile["angmom_ratio"]),
                com_height_ratio=float(profile["com_height_ratio"]),
                asymmetry=float(profile["asymmetry"]),
                precision_h=float(profile["precision_H"]),
                precision_t=float(profile["precision_T"]),
                precision_l=float(profile["precision_L"]),
                support_mode=str(profile["support_mode"]),
                single_support_ratio=float(profile["single_support_ratio"]),
                com_drop_m=float(profile["com_drop_m"]),
                support_margin_m=float(profile["support_margin_m"]),
                torso_pitch_comp_rad=float(profile["torso_pitch_comp_rad"]),
                torso_roll_comp_rad=float(profile["torso_roll_comp_rad"]),
                torso_yaw_comp_rad=float(profile["torso_yaw_comp_rad"]),
                arm_counter_pitch_rad=float(profile["arm_counter_pitch_rad"]),
                arm_counter_roll_rad=float(profile["arm_counter_roll_rad"]),
                arm_counter_yaw_rad=float(profile["arm_counter_yaw_rad"]),
                knee_flex_ratio=float(profile["knee_flex_ratio"]),
                stability_score=float(profile["stability_score"]),
            )
        )
    return actions


def speed_reference(ctx: RobotContext, category: str) -> float:
    if category == "punch":
        return ctx.v_ref_punch
    if category == "kick":
        return ctx.v_ref_kick
    if category == "combo":
        return ctx.v_ref_combo
    return ctx.v_ref_special


def radius_reference(ctx: RobotContext, category: str) -> float:
    if category == "punch":
        return ctx.r_ref_punch
    if category == "kick":
        return ctx.r_ref_kick
    if category == "combo":
        return ctx.r_ref_combo
    return ctx.r_ref_special


def compute_raw_metrics(ctx: RobotContext, actions: List[ActionPortrait]) -> List[Dict[str, float]]:
    rows: List[Dict[str, float]] = []
    for action in actions:
        v_ref = speed_reference(ctx, action.category)
        radius = radius_reference(ctx, action.category)
        m_eff = ctx.total_mass_kg * action.m_eff_ratio
```

```
        v_strike = max(0.5, v_ref * action.speed_util)
        p_linear = m_eff * v_strike
        e_kinetic = 0.5 * m_eff * v_strike * v_strike
        inertia = 0.35 * m_eff * radius * radius
        p_angular = inertia * v_strike / max(radius, 1e-6)

        energy_joule = (
            (
                action.q25_count * ctx.light_effort_nm * action.excursion_q25
                + action.q90_count * ctx.heavy_effort_nm * action.excursion_q90
            )
            * action.torque_util
            * (1.0 + 0.30 * action.speed_util)
            * action.duration_s
        )

        disturbance_index = (
            0.30 * action.zmp_ratio
            + 0.24 * action.angmom_ratio
            + 0.12 * action.com_height_ratio
            + 0.16 * action.single_support_ratio
            + 0.10 * action.asymmetry
            + SUPPORT_MODE_PENALTY[action.support_mode]
        )

        compensation_index = (
            0.22 * (action.support_margin_m / max(ctx.foot_width_m, 1e-6))
            + 0.18 * (action.com_drop_m / max(ctx.leg_total_m, 1e-6))
            + 0.18 * action.knee_flex_ratio
            + 0.10 * (action.torso_pitch_comp_rad + action.torso_roll_comp_rad + action.torso_yaw_comp_rad)
            + 0.10 * (action.arm_counter_pitch_rad + action.arm_counter_roll_rad + action.arm_counter_yaw_rad)
        )

        rows.append(
            {
                "action_key": action.action_key,
                "action_name_cn": action.action_name_cn,
                "m_eff_kg": m_eff,
                "v_strike_m_per_s": v_strike,
                "p_linear": p_linear,
                "e_kinetic": e_kinetic,
                "p_angular": p_angular,
                "energy_joule": energy_joule,
                "disturbance_index": disturbance_index,
                "compensation_index": compensation_index,
            }
        )
    return rows


def add_normalized_fields(rows: List[Dict[str, float]], keys: List[str]) -> None:
    for key in keys:
        max_value = max(row[key] for row in rows)
        for row in rows:
            row[f"{key}_norm"] = row[key] / max_value if max_value > 0 else 0.0


def site_precision(action: ActionPortrait, site: str) -> float:
    if site == "H":
        return action.precision_h
    if site == "T":
        return action.precision_t
    return action.precision_l


def stability_reserve(action: ActionPortrait, row: Dict[str, float]) -> float:
    reserve_proxy = clamp(
        0.55 + 1.10 * (row["compensation_index_norm"] - row["disturbance_index_norm"]),
        0.0,
        1.0,
    )
    return clamp(0.60 * action.stability_score + 0.40 * reserve_proxy, 0.0, 1.0)


def support_penalty(action: ActionPortrait) -> float:
    penalty = 0.24 * action.single_support_ratio
    if action.support_mode == "transition_support":
        penalty += 0.08
    elif action.support_mode == "alternating_support":
        penalty += 0.12
    return penalty


def compute_ranking(ctx: RobotContext, actions: List[ActionPortrait]) -> Tuple[List[Dict[str, object]], Dict[str, List[Dict[str, object]]]]:
    raw_rows = compute_raw_metrics(ctx, actions)
    add_normalized_fields(
        raw_rows,
        ["p_linear", "e_kinetic", "p_angular", "energy_joule", "disturbance_index", "compensation_index"],
    )
    raw_by_key = {row["action_key"]: row for row in raw_rows}
```

```python
    ranking: List[Dict[str, object]] = []
    site_details: Dict[str, List[Dict[str, object]]] = {}

    torso_height = SITE_HEIGHT["T"]
    for action in actions:
        raw = raw_by_key[action.action_key]
        strike_base = (
            0.34 * raw["p_linear_norm"]
            + 0.41 * raw["e_kinetic_norm"]
            + 0.25 * raw["p_angular_norm"]
        ) * (1.0 + 0.05 * math.log(ACTION_HITS[action.action_key]))
        reserve = stability_reserve(action, raw)
        risk = 1.0 - reserve
        energy_cost = raw["energy_joule_norm"]

        per_site: List[Dict[str, object]] = []
        for site in ("H", "T", "L"):
            precision = site_precision(action, site)
            if precision <= 0:
                continue
            strike_site = strike_base * SITE_DAMAGE[site] * SITE_EXPOSURE[site] * precision
            height_pen = 0.18 * max(0.0, (SITE_HEIGHT[site] - torso_height) / torso_height)
            score = (
                0.50 * strike_site
                + 0.34 * reserve
                - 0.16 * energy_cost
                - 0.22 * risk
                - height_pen
                - support_penalty(action)
            ) * TACTICAL_AVAILABILITY[action.action_key]

            per_site.append(
                {
                    "site": site,
                    "site_name_cn": SITE_NAME_CN[site],
                    "precision": precision,
                    "strike_site": strike_site,
                    "stability_reserve": reserve,
                    "energy_cost": energy_cost,
                    "height_penalty": height_pen,
                    "support_penalty": support_penalty(action),
                    "tactical_availability": TACTICAL_AVAILABILITY[action.action_key],
                    "score": score,
                }
            )

        best = max(per_site, key=lambda item: item["score"])
        site_details[action.action_key] = per_site
        ranking.append(
            {
                "action_key": action.action_key,
                "action_name_cn": action.action_name_cn,
                "category": action.category,
                "best_site": best["site"],
                "best_site_name_cn": best["site_name_cn"],
                "q_star": best["score"],
                "strike_base": strike_base,
                "strike_best_site": best["strike_site"],
                "stability_reserve": best["stability_reserve"],
                "energy_cost": best["energy_cost"],
                "tactical_availability": best["tactical_availability"],
                "support_mode": action.support_mode,
                "single_support_ratio": action.single_support_ratio,
                "stability_score": action.stability_score,
                "m_eff_kg": raw["m_eff_kg"],
                "v_strike_m_per_s": raw["v_strike_m_per_s"],
                "p_linear": raw["p_linear"],
                "e_kinetic": raw["e_kinetic"],
                "p_angular": raw["p_angular"],
            }
        )

    ranking.sort(key=lambda item: item["q_star"], reverse=True)
    for idx, row in enumerate(ranking, start=1):
        row["rank"] = idx
    return ranking, site_details


def top_action_frequencies(ranking: List[Dict[str, object]], site_details: Dict[str, List[Dict[str, object]]], samples: int = 1000) -> List[
      Dict[str, object]]:
    rng = np.random.default_rng(20260419)
    keys = [row["action_key"] for row in ranking]
    base_weights = np.array([0.50, 0.34, 0.16, 0.22], dtype=float)
    count = {key: 0 for key in keys}

    for _ in range(samples):
        multipliers = rng.uniform(0.90, 1.10, size=4)
        weights = base_weights * multipliers
        strike_w, stab_w, energy_w, risk_w = weights

        perturbed_scores: List[Tuple[str, float]] = []
```

```python
        for row in ranking:
            action_key = str(row["action_key"])
            best_score = None
            for detail in site_details[action_key]:
                score = (
                    strike_w * float(detail["strike_site"])
                    + stab_w * float(detail["stability_reserve"])
                    - energy_w * float(detail["energy_cost"])
                    - risk_w * (1.0 - float(detail["stability_reserve"]))
                    - float(detail["height_penalty"])
                    - float(detail["support_penalty"])
                ) * float(detail["tactical_availability"])
                if best_score is None or score > best_score:
                    best_score = score
            perturbed_scores.append((action_key, float(best_score)))

        perturbed_scores.sort(key=lambda item: item[1], reverse=True)
        for action_key, _ in perturbed_scores[:4]:
            count[action_key] += 1

    output: List[Dict[str, object]] = []
    for row in ranking:
        action_key = str(row["action_key"])
        output.append(
            {
                "action_key": action_key,
                "action_name_cn": row["action_name_cn"],
                "top4_frequency": count[action_key] / samples,
            }
        )
    return output


def build_summary(ranking: List[Dict[str, object]]) -> Dict[str, object]:
    primary = [
        row["action_key"]
        for row in ranking
        if float(row["tactical_availability"]) >= 0.85 and row["support_mode"] != "multi_contact_ground"
    ][:4]
    situational = [
        row["action_key"]
        for row in ranking
        if row["action_key"] not in primary and float(row["q_star"]) > 0.10
    ][:4]
    best_site_count = {"H": 0, "T": 0, "L": 0}
    for row in ranking:
        best_site_count[str(row["best_site"])] += 1
    return {
        "recommended_primary_actions": primary,
        "recommended_situational_actions": situational,
        "best_site_distribution": best_site_count,
    }


def write_csv(path: Path, rows: List[Dict[str, object]], fieldnames: List[str]) -> None:
    with path.open("w", encoding="utf-8-sig", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)


def write_json(path: Path, payload: Dict[str, object]) -> None:
```

问题二核心代码

```python
#!/usr/bin/env python3
"""Solve MathorCup B Question 2 using PM01 balance-aware defense portraits."""

from __future__ import annotations

import csv
import json
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Tuple

import matplotlib
import numpy as np
from matplotlib import pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

matplotlib.use("Agg")

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False


WORKSPACE_ROOT = Path(__file__).resolve().parents[1]
```

```
Q1_RANKING_JSON = WORKSPACE_ROOT / "outputs" / "q1_complete_solution" / "q1_priority_ranking.json"
Q1_PORTRAITS_JSON = WORKSPACE_ROOT / "data" / "question1_action_portraits" / "q1_action_portraits_pm01.json"
Q2_DEFENSE_JSON = WORKSPACE_ROOT / "data" / "question2_defense_portraits" / "q2_defense_portraits_pm01.json"
OUTPUT_DIR = WORKSPACE_ROOT / "outputs" / "q2_complete_solution"


WEIGHTS = {
    "residual_damage": 0.44,
    "balance_burden": 0.20,
    "energy_cost": 0.12,
    "timing_penalty": 0.14,
    "sequence_penalty": 0.12,
    "counter_bonus": 0.08,
}


SUPPORT_MODE_PENALTY = {
    "double_support": 0.00,
    "lowered_double_support": 0.04,
    "step_transition": 0.10,
    "rotating_transition": 0.13,
    "single_support": 0.16,
    "multi_contact_ground": -0.08,
}


TRAJECTORY_URGENCY = {
    "linear": 0.86,
    "hook": 0.80,
    "spin": 0.72,
    "sweep": 0.76,
    "charge": 0.90,
    "combo": 0.88,
    "counter": 0.62,
}


ATTACK_META = {
    "straight_punch": {"trajectory": "linear", "hits": 1},
    "hook_punch": {"trajectory": "hook", "hits": 1},
    "combo_punch": {"trajectory": "combo", "hits": 2},
    "swing_punch": {"trajectory": "hook", "hits": 1},
    "front_kick": {"trajectory": "linear", "hits": 1},
    "side_kick": {"trajectory": "linear", "hits": 1},
    "spin_kick": {"trajectory": "spin", "hits": 1},
    "low_sweep": {"trajectory": "sweep", "hits": 1},
    "knee_strike": {"trajectory": "charge", "hits": 1},
    "punch_kick_combo": {"trajectory": "combo", "hits": 2},
    "five_kick_combo": {"trajectory": "combo", "hits": 5},
    "charge": {"trajectory": "charge", "hits": 1},
    "ground_counter": {"trajectory": "counter", "hits": 2},
}


@dataclass(frozen=True)
class AttackState:
    action_key: str
    action_name_cn: str
    category: str
    trajectory: str
    hits: int
    best_site: str
    strike_value: float
    strike_base: float
    v_strike_m_per_s: float
    duration_s: float
    m_eff_ratio: float
    zmp_ratio: float
    angmom_ratio: float
    com_height_ratio: float
    support_mode: str
    single_support_ratio: float
    stability_score: float
    precision_h: float
    precision_t: float
    precision_l: float


@dataclass(frozen=True)
class DefenseState:
    defense_key: str
    defense_name_cn: str
    style: str
    mode: str
    cover_h: float
    cover_t: float
    cover_l: float
    anti_linear: float
    anti_hook: float
    anti_spin: float
    anti_low: float
```

```
    anti_charge: float
    anti_combo: float
    mech_absorb: float
    evade_h: float
    evade_t: float
    evade_l: float
    evade_lateral: float
    evade_retreat: float
    evade_rotate: float
    q25_count: int
    q90_count: int
    torque_util: float
    excursion_q25: float
    excursion_q90: float
    duration_s: float
    support_mode: str
    support_focus: str
    single_support_ratio: float
    stance_width_m: float
    com_shift_sagittal_m: float
    com_shift_lateral_m: float
    com_drop_m: float
    torso_pitch_comp_rad: float
    torso_roll_comp_rad: float
    torso_yaw_comp_rad: float
    arm_guard_pitch_rad: float
    arm_guard_roll_rad: float
    arm_balance_yaw_rad: float
    knee_flex_ratio: float
    support_margin_m: float
    recovery_s: float
    transition_s: float
    raw_stability_risk: float
    compensation_efficiency: float
    balance_reserve_score: float
    stability_score: float
    timing_agility_score: float
    counter_window_score: float
    mechanical_energy_j: float


def clamp(value: float, lower: float, upper: float) -> float:
    return max(lower, min(upper, value))


def round_num(value: float, digits: int = 6) -> float:
    return round(float(value), digits)


def load_json(path: Path) -> Dict[str, object]:
    with path.open("r", encoding="utf-8") as handle:
        return json.load(handle)


def load_attacks() -> Tuple[List[AttackState], Dict[str, float]]:
    ranking_payload = load_json(Q1_RANKING_JSON)
    portrait_payload = load_json(Q1_PORTRAITS_JSON)
    portrait_map = {
        item["pm01_calibrated_profile"]["action_key"]: item["pm01_calibrated_profile"]
        for item in portrait_payload["actions"]
    }

    attacks: List[AttackState] = []
    momentum_rows: List[Tuple[str, float, float, float]] = []
    for row in ranking_payload["ranking"]:
        action_key = str(row.get("action_key") or row.get("action") or "").strip()
        if not action_key:
            continue
        meta = ATTACK_META[action_key]
        portrait = portrait_map[action_key]
        attacks.append(
            AttackState(
                action_key=action_key,
                action_name_cn=str(row["action_name_cn"]),
                category=str(row["category"]),
                trajectory=str(meta["trajectory"]),
                hits=int(meta["hits"]),
                best_site=str(row["best_site"]),
                strike_value=float(row["strike_best_site"]),
                strike_base=float(row["strike_base"]),
                v_strike_m_per_s=float(row["v_strike_m_per_s"]),
                duration_s=float(portrait["duration_s"]),
                m_eff_ratio=float(portrait["m_eff_ratio"]),
                zmp_ratio=float(portrait["zmp_ratio"]),
                angmom_ratio=float(portrait["angmom_ratio"]),
                com_height_ratio=float(portrait["com_height_ratio"]),
                support_mode=str(portrait["support_mode"]),
                single_support_ratio=float(portrait["single_support_ratio"]),
                stability_score=float(portrait["stability_score"]),
                precision_h=float(portrait["precision_H"]),
                precision_t=float(portrait["precision_T"]),
```

```
                precision_l=float(portrait["precision_L"]),
            )
        )
        momentum_proxy = float(portrait["m_eff_ratio"]) * float(row["v_strike_m_per_s"])
        hit_density = int(meta["hits"]) / max(float(portrait["duration_s"]), 1e-6)
        balance_pressure = (
            0.30 * float(portrait["zmp_ratio"])
            + 0.28 * float(portrait["angmom_ratio"])
            + 0.16 * float(portrait["single_support_ratio"])
            + 0.12 * clamp((int(meta["hits"]) - 1) / 4.0, 0.0, 1.0)
            + 0.08 * (1.0 - float(portrait["stability_score"]))
            + 0.06 * float(portrait["com_height_ratio"])
        )
        momentum_rows.append((action_key, momentum_proxy, hit_density, balance_pressure))

    max_strike = max(a.strike_value for a in attacks)
    max_speed = max(a.v_strike_m_per_s for a in attacks)
    max_momentum = max(item[1] for item in momentum_rows)
    max_hit_density = max(item[2] for item in momentum_rows)
    max_balance_pressure = max(item[3] for item in momentum_rows)

    norms = {}
    for action_key, momentum_proxy, hit_density, balance_pressure in momentum_rows:
        norms[f"{action_key}.momentum_norm"] = momentum_proxy / max_momentum
        norms[f"{action_key}.hit_density_norm"] = hit_density / max_hit_density
        norms[f"{action_key}.balance_pressure_norm"] = balance_pressure / max_balance_pressure
    norms["max_strike"] = max_strike
    norms["max_speed"] = max_speed
    return attacks, norms


def load_defenses() -> Tuple[List[DefenseState], Dict[str, float]]:
    payload = load_json(Q2_DEFENSE_JSON)
    defenses: List[DefenseState] = []
    balance_costs: List[Tuple[str, float, float]] = []
    foot_width = float(payload["robot_basis"]["foot_width_m"])
    for row in payload["defenses"]:
        defense = DefenseState(
            defense_key=str(row["defense_key"]),
            defense_name_cn=str(row["defense_name_cn"]),
            style=str(row["style"]),
            mode=str(row["mode"]),
            cover_h=float(row["cover_h"]),
            cover_t=float(row["cover_t"]),
            cover_l=float(row["cover_l"]),
            anti_linear=float(row["anti_linear"]),
            anti_hook=float(row["anti_hook"]),
            anti_spin=float(row["anti_spin"]),
            anti_low=float(row["anti_low"]),
            anti_charge=float(row["anti_charge"]),
            anti_combo=float(row["anti_combo"]),
            mech_absorb=float(row["mech_absorb"]),
            evade_h=float(row["evade_h"]),
            evade_t=float(row["evade_t"]),
            evade_l=float(row["evade_l"]),
            evade_lateral=float(row["evade_lateral"]),
            evade_retreat=float(row["evade_retreat"]),
            evade_rotate=float(row["evade_rotate"]),
            q25_count=int(row["q25_count"]),
            q90_count=int(row["q90_count"]),
            torque_util=float(row["torque_util"]),
            excursion_q25=float(row["excursion_q25"]),
            excursion_q90=float(row["excursion_q90"]),
            duration_s=float(row["duration_s"]),
            support_mode=str(row["support_mode"]),
            support_focus=str(row["support_focus"]),
            single_support_ratio=float(row["single_support_ratio"]),
            stance_width_m=float(row["stance_width_m"]),
            com_shift_sagittal_m=float(row["com_shift_sagittal_m"]),
            com_shift_lateral_m=float(row["com_shift_lateral_m"]),
            com_drop_m=float(row["com_drop_m"]),
            torso_pitch_comp_rad=float(row["torso_pitch_comp_rad"]),
            torso_roll_comp_rad=float(row["torso_roll_comp_rad"]),
            torso_yaw_comp_rad=float(row["torso_yaw_comp_rad"]),
            arm_guard_pitch_rad=float(row["arm_guard_pitch_rad"]),
            arm_guard_roll_rad=float(row["arm_guard_roll_rad"]),
            arm_balance_yaw_rad=float(row["arm_balance_yaw_rad"]),
            knee_flex_ratio=float(row["knee_flex_ratio"]),
            support_margin_m=float(row["support_margin_m"]),
            recovery_s=float(row["recovery_s"]),
            transition_s=float(row["transition_s"]),
            raw_stability_risk=float(row["raw_stability_risk"]),
            compensation_efficiency=float(row["compensation_efficiency"]),
            balance_reserve_score=float(row["balance_reserve_score"]),
            stability_score=float(row["stability_score"]),
            timing_agility_score=float(row["timing_agility_score"]),
            counter_window_score=float(row["counter_window_score"]),
            mechanical_energy_j=float(row["mechanical_energy_j"]),
        )
        defenses.append(defense)
        support_margin_ratio = defense.support_margin_m / max(foot_width, 1e-6)
```

```
        base_balance_cost = clamp(
            0.42 * defense.raw_stability_risk
            + 0.18 * defense.single_support_ratio
            + 0.12 * defense.torque_util
            + 0.10 * defense.excursion_q90
            + 0.08 * defense.excursion_q25
            + 0.10 * SUPPORT_MODE_PENALTY[defense.support_mode]
            - 0.22 * defense.balance_reserve_score
            - 0.10 * support_margin_ratio,
            0.02,
            1.10,
        )
        balance_costs.append((defense.defense_key, base_balance_cost, defense.mechanical_energy_j))

    max_energy = max(item[2] for item in balance_costs)
    context = {}
    for defense_key, base_balance_cost, energy in balance_costs:
        context[f"{defense_key}.base_balance_cost"] = base_balance_cost
        context[f"{defense_key}.energy_norm"] = energy / max_energy if max_energy > 0 else 0.0
    context["foot_width_m"] = foot_width
    return defenses, context


def site_cover(defense: DefenseState, site: str) -> float:
    if site == "H":
        return defense.cover_h
    if site == "T":
        return defense.cover_t
    return defense.cover_l


def site_evade(defense: DefenseState, site: str) -> float:
    if site == "H":
        return defense.evade_h
    if site == "T":
        return defense.evade_t
    return defense.evade_l


def trajectory_block(defense: DefenseState, trajectory: str) -> float:
    if trajectory == "linear":
        return defense.anti_linear
    if trajectory == "hook":
        return defense.anti_hook
    if trajectory == "spin":
        return defense.anti_spin
    if trajectory == "sweep":
        return defense.anti_low
    if trajectory == "charge":
        return defense.anti_charge
    if trajectory == "combo":
        return defense.anti_combo
    return 0.5 * (defense.anti_linear + defense.anti_combo)


def trajectory_evade(defense: DefenseState, trajectory: str) -> float:
    if trajectory == "linear":
        return 0.46 * defense.evade_lateral + 0.38 * defense.evade_retreat + 0.16 * defense.evade_rotate
    if trajectory == "hook":
        return 0.48 * defense.evade_lateral + 0.20 * defense.evade_retreat + 0.32 * defense.evade_rotate
    if trajectory == "spin":
        return 0.18 * defense.evade_lateral + 0.24 * defense.evade_retreat + 0.58 * defense.evade_rotate
    if trajectory == "sweep":
        return 0.18 * defense.evade_lateral + 0.40 * defense.evade_retreat + 0.18 * defense.evade_rotate + 0.24 * defense.evade_l
    if trajectory == "charge":
        return 0.20 * defense.evade_lateral + 0.60 * defense.evade_retreat + 0.20 * defense.evade_rotate
    if trajectory == "combo":
        return 0.34 * defense.evade_lateral + 0.36 * defense.evade_retreat + 0.30 * defense.evade_rotate
    return 0.32 * defense.evade_lateral + 0.28 * defense.evade_retreat + 0.40 * defense.evade_rotate


def style_bonus(attack: AttackState, defense: DefenseState) -> float:
    bonus = 0.0
    if attack.trajectory == "charge" and defense.style in {"balance", "ground"}:
        bonus += 0.06
    if attack.trajectory == "sweep" and defense.cover_l >= 0.70:
        bonus += 0.06
    if attack.trajectory == "spin" and max(defense.anti_spin, defense.evade_rotate) >= 0.75:
        bonus += 0.06
    if attack.hits >= 2 and defense.anti_combo >= 0.74 and defense.recovery_s <= 0.28:
        bonus += 0.06
    if attack.action_key == "ground_counter" and defense.style == "ground":
        bonus += 0.10
    return bonus


def attack_specific_bonus(attack: AttackState, defense: DefenseState) -> float:
    if attack.action_key == "front_kick" and defense.defense_key in {"backstep", "side_body_stance", "step_adjust", "low_center_stance"}:
        return 0.08
    if attack.action_key == "side_kick" and defense.defense_key in {"lateral_dodge", "side_body_stance", "sidestep_circle", "step_adjust"}:
        return 0.10
```

```
    if attack.action_key == "spin_kick" and defense.defense_key in {"spin_evade", "sidestep_circle", "lateral_dodge"}:
        return 0.14
    if attack.action_key == "five_kick_combo" and defense.defense_key in {"block_retreat_circle", "sidestep_circle", "step_adjust", "
      lateral_dodge"}:
        return 0.16
    if attack.action_key == "punch_kick_combo" and defense.defense_key in {"evade_block_counter", "block_retreat_circle", "step_adjust", "
      side_body_stance"}:
        return 0.12
    if attack.action_key == "low_sweep" and defense.defense_key in {"downward_block", "low_center_stance", "step_adjust"}:
        return 0.10
    if attack.action_key == "charge" and defense.defense_key in {"impact_buffer", "com_compensation", "backstep", "controlled_fall"}:
        return 0.10
    if attack.action_key == "ground_counter" and defense.defense_key in {"grounded_guard", "controlled_fall", "quick_getup"}:
        return 0.14
    return 0.0


def attack_specific_penalty(attack: AttackState, defense: DefenseState) -> float:
    if attack.action_key == "front_kick":
        required = max(defense.anti_linear, defense.evade_retreat, defense.knee_flex_ratio + 0.40 * defense.balance_reserve_score)
        penalty = 0.18 * max(0.0, 0.72 - required)
    elif attack.action_key == "side_kick":
        required = max(defense.evade_lateral, defense.anti_spin, 0.45 * defense.cover_t + 0.55 * defense.balance_reserve_score)
        penalty = 0.20 * max(0.0, 0.74 - required)
    elif attack.action_key == "spin_kick":
        required = max(defense.anti_spin, defense.evade_rotate, defense.evade_lateral)
        penalty = 0.26 * max(0.0, 0.80 - required)
    elif attack.action_key == "low_sweep":
        required = max(defense.anti_low, defense.cover_l, defense.knee_flex_ratio + 0.50 * defense.balance_reserve_score)
        penalty = 0.22 * max(0.0, 0.78 - required)
    elif attack.action_key == "charge":
        required = max(defense.anti_charge, defense.mech_absorb, defense.balance_reserve_score)
        penalty = 0.22 * max(0.0, 0.80 - required)
    elif attack.action_key == "ground_counter":
        required = max(defense.cover_l, defense.evade_retreat, 1.0 if defense.style == "ground" else 0.0)
        penalty = 0.18 * max(0.0, 0.74 - required)
        if defense.style != "ground":
            penalty += 0.08
    elif attack.action_key == "five_kick_combo":
        required = max(defense.anti_combo, defense.evade_lateral, defense.evade_retreat, defense.balance_reserve_score)
        penalty = 0.24 * max(0.0, 0.80 - required)
        if defense.style in {"block", "posture"} and defense.support_mode in {"double_support", "lowered_double_support"}:
            penalty += 0.06
    elif attack.action_key == "punch_kick_combo":
        required = max(defense.anti_combo, defense.evade_lateral, defense.balance_reserve_score, 1.0 - defense.recovery_s)
        penalty = 0.18 * max(0.0, 0.74 - required)
    elif attack.hits > 1:
        required = max(defense.anti_combo, 1.0 - defense.recovery_s, defense.timing_agility_score)
        penalty = 0.18 * max(0.0, 0.72 - required)
    elif attack.category == "kick":
        required = max(defense.evade_lateral, defense.evade_retreat, defense.anti_linear, defense.balance_reserve_score)
        penalty = 0.10 * max(0.0, 0.64 - required)
    else:
        penalty = 0.0

    if attack.category == "kick" and defense.style == "block" and defense.support_mode == "double_support" and defense.knee_flex_ratio < 0.22:
        penalty += 0.04
    return clamp(penalty, 0.0, 0.32)


def build_pairwise_rows(attacks: List[AttackState], norms: Dict[str, float], defenses: List[DefenseState], defense_ctx: Dict[str, float]) ->
      List[Dict[str, object]]:
    rows: List[Dict[str, object]] = []
    max_strike = norms["max_strike"]
    max_speed = norms["max_speed"]
    foot_width = defense_ctx["foot_width_m"]

    for attack in attacks:
        strike_norm = attack.strike_value / max_strike
        speed_norm = attack.v_strike_m_per_s / max_speed
        momentum_norm = norms[f"{attack.action_key}.momentum_norm"]
        hit_density_norm = norms[f"{attack.action_key}.hit_density_norm"]
        balance_pressure = norms[f"{attack.action_key}.balance_pressure_norm"]
        urgency = clamp(
            0.40 * speed_norm
            + 0.20 * hit_density_norm
            + 0.22 * TRAJECTORY_URGENCY[attack.trajectory]
            + 0.18 * strike_norm,
            0.08,
            1.00,
        )
        attack_window = attack.duration_s / max(attack.hits, 1)

        for defense in defenses:
            zone_cover = site_cover(defense, attack.best_site)
            zone_evade = site_evade(defense, attack.best_site)
            block_score = clamp(0.56 * zone_cover + 0.44 * trajectory_block(defense, attack.trajectory), 0.0, 0.98)
            evade_score = clamp(0.42 * zone_evade + 0.58 * trajectory_evade(defense, attack.trajectory), 0.0, 0.98)
            mechanical_buffer = clamp(
                0.72 * defense.mech_absorb + 0.14 * defense.knee_flex_ratio + 0.14 * defense.balance_reserve_score,
                0.0,
```

```
    0.98,
)
support_margin_ratio = defense.support_margin_m / max(foot_width, 1e-6)
stability_support = clamp(
    0.42 * defense.stability_score
    + 0.22 * defense.balance_reserve_score
    + 0.18 * defense.compensation_efficiency
    + 0.10 * support_margin_ratio
    + 0.08 * (1.0 - defense.single_support_ratio),
    0.0,
    0.98,
)
defense_response = 0.60 * defense.duration_s + 0.40 * defense.transition_s
timing_match = clamp(
    1.06 - urgency * defense_response / max(attack_window * 1.12, 0.18),
    0.02,
    0.98,
)

if defense.mode == "block":
    intercept = 0.48 * block_score + 0.24 * mechanical_buffer + 0.16 * timing_match + 0.12 * stability_support
elif defense.mode == "evade":
    intercept = 0.54 * evade_score + 0.24 * timing_match + 0.22 * stability_support
else:
    intercept = (
        0.28 * block_score
        + 0.24 * evade_score
        + 0.16 * mechanical_buffer
        + 0.16 * timing_match
        + 0.16 * stability_support
    )

mismatch_penalty = clamp(
    0.16 * max(0.0, 0.45 - zone_cover)
    + 0.14 * max(0.0, 0.45 - trajectory_block(defense, attack.trajectory))
    + 0.06 * max(0.0, 0.40 - zone_evade),
    0.0,
    0.18,
)
specific_penalty = attack_specific_penalty(attack, defense)
intercept = clamp(
    intercept
    + style_bonus(attack, defense)
    + attack_specific_bonus(attack, defense)
    - mismatch_penalty
    - 0.60 * specific_penalty,
    0.02,
    0.98,
)

residual_damage = clamp(
    strike_norm * (1.0 - intercept) * (1.0 + 0.22 * momentum_norm + 0.10 * max(0, attack.hits - 1)),
    0.0,
    1.50,
)
base_balance_cost = defense_ctx[f"{defense.defense_key}.base_balance_cost"]
balance_burden = clamp(
    0.62 * base_balance_cost * (1.0 + 0.30 * balance_pressure + 0.18 * momentum_norm)
    + 0.10 * defense.single_support_ratio
    + 0.08 * max(0.0, urgency - defense.timing_agility_score)
    + 0.50 * specific_penalty
    - 0.10 * stability_support,
    0.02,
    1.20,
)
timing_penalty = clamp((1.0 - timing_match) * (0.78 + 0.22 * urgency) + 0.35 * specific_penalty, 0.0, 1.00)
if attack.hits > 1:
    overflow = max(0.0, defense.recovery_s + 0.65 * defense.transition_s - attack_window)
    sequence_penalty = clamp(
        overflow / max(attack_window, 1e-6) * (0.56 + 0.10 * attack.hits)
        + 0.12 * (1.0 - defense.anti_combo)
        + 0.20 * specific_penalty,
        0.0,
        1.40,
    )
else:
    sequence_penalty = 0.18 * specific_penalty if attack.trajectory in {"spin", "charge", "counter"} else 0.0
counter_bonus = clamp(
    defense.counter_window_score
    * intercept
    * (0.55 + 0.20 * (1.0 - balance_pressure))
    * (0.90 if attack.hits > 2 else 1.0),
    0.0,
    0.60,
)
energy_cost = defense_ctx[f"{defense.defense_key}.energy_norm"]
total_loss = (
    WEIGHTS["residual_damage"] * residual_damage
    + WEIGHTS["balance_burden"] * balance_burden
    + WEIGHTS["energy_cost"] * energy_cost
    + WEIGHTS["timing_penalty"] * timing_penalty
```

```
            + WEIGHTS["sequence_penalty"] * sequence_penalty
            - WEIGHTS["counter_bonus"] * counter_bonus
        )

        rows.append(
            {
                "attack_key": attack.action_key,
                "attack_name_cn": attack.action_name_cn,
                "attack_category": attack.category,
                "attack_trajectory": attack.trajectory,
                "attack_best_site": attack.best_site,
                "defense_key": defense.defense_key,
                "defense_name_cn": defense.defense_name_cn,
                "defense_style": defense.style,
                "defense_mode": defense.mode,
                "zone_cover": round_num(zone_cover),
                "zone_evade": round_num(zone_evade),
                "block_score": round_num(block_score),
                "evade_score": round_num(evade_score),
                "mechanical_buffer": round_num(mechanical_buffer),
                "stability_support": round_num(stability_support),
                "timing_match": round_num(timing_match),
                "intercept_score": round_num(intercept),
                "residual_damage": round_num(residual_damage),
                "balance_burden": round_num(balance_burden),
                "energy_cost": round_num(energy_cost),
                "timing_penalty": round_num(timing_penalty),
                "sequence_penalty": round_num(sequence_penalty),
                "counter_bonus": round_num(counter_bonus),
                "total_loss": round_num(total_loss),
            }
        )
    return rows


def summarize_best(rows: List[Dict[str, object]], delta: float = 0.06) -> Tuple[List[Dict[str, object]], Dict[str, List[Dict[str, object]]]]:
    grouped: Dict[str, List[Dict[str, object]]] = {}
    for row in rows:
        grouped.setdefault(str(row["attack_key"]), []).append(row)

    summary_rows: List[Dict[str, object]] = []
    topk_details: Dict[str, List[Dict[str, object]]] = {}
    for attack_key, attack_rows in grouped.items():
        sorted_rows = sorted(attack_rows, key=lambda item: float(item["total_loss"]))
        best = sorted_rows[0]
        near_opt = [row for row in sorted_rows if float(row["total_loss"]) <= float(best["total_loss"]) + delta]
        topk_details[attack_key] = sorted_rows[:5]
        summary_rows.append(
            {
                "attack_key": attack_key,
                "attack_name_cn": best["attack_name_cn"],
                "attack_best_site": best["attack_best_site"],
                "best_defense_key": best["defense_key"],
                "best_defense_name_cn": best["defense_name_cn"],
                "best_style": best["defense_style"],
                "best_loss": round_num(best["total_loss"]),
                "second_defense_key": sorted_rows[1]["defense_key"],
                "second_defense_name_cn": sorted_rows[1]["defense_name_cn"],
                "second_loss": round_num(sorted_rows[1]["total_loss"]),
                "third_defense_key": sorted_rows[2]["defense_key"],
                "third_defense_name_cn": sorted_rows[2]["defense_name_cn"],
                "third_loss": round_num(sorted_rows[2]["total_loss"]),
                "near_opt_keys": "|".join(str(row["defense_key"]) for row in near_opt),
                "near_opt_names_cn": "|".join(str(row["defense_name_cn"]) for row in near_opt),
                "near_opt_count": len(near_opt),
            }
        )
    summary_rows.sort(key=lambda item: item["attack_key"])
    return summary_rows, topk_details


def compute_robustness(rows: List[Dict[str, object]], summary_rows: List[Dict[str, object]], samples: int = 1200) -> List[Dict[str, object]]:
    grouped: Dict[str, List[Dict[str, object]]] = {}
    for row in rows:
        grouped.setdefault(str(row["attack_key"]), []).append(row)

    rng = np.random.default_rng(20260419)
    base_weights = np.array(
        [
            WEIGHTS["residual_damage"],
            WEIGHTS["balance_burden"],
            WEIGHTS["energy_cost"],
            WEIGHTS["timing_penalty"],
            WEIGHTS["sequence_penalty"],
            WEIGHTS["counter_bonus"],
        ],
        dtype=float,
    )
    best_counter = {row["attack_key"]: 0 for row in summary_rows}
    near_counter = {row["attack_key"]: 0 for row in summary_rows}
```

```python
    for _ in range(samples):
        multipliers = rng.uniform(0.88, 1.12, size=6)
        weights = base_weights * multipliers
        for attack_key, attack_rows in grouped.items():
            scored = []
            for row in attack_rows:
                score = (
                    weights[0] * float(row["residual_damage"])
                    + weights[1] * float(row["balance_burden"])
                    + weights[2] * float(row["energy_cost"])
                    + weights[3] * float(row["timing_penalty"])
                    + weights[4] * float(row["sequence_penalty"])
                    - weights[5] * float(row["counter_bonus"])
                )
                scored.append((str(row["defense_key"]), score))
            scored.sort(key=lambda item: item[1])
            best_counter[attack_key] += int(scored[0][0] == next(row["best_defense_key"] for row in summary_rows if row["attack_key"] ==
      attack_key))
            top3 = {name for name, _ in scored[:3]}
            near_counter[attack_key] += int(next(row["best_defense_key"] for row in summary_rows if row["attack_key"] == attack_key) in top3)

    output = []
    for row in summary_rows:
        output.append(
            {
                "attack_key": row["attack_key"],
                "attack_name_cn": row["attack_name_cn"],
                "best_defense_key": row["best_defense_key"],
                "best_defense_name_cn": row["best_defense_name_cn"],
                "top1_frequency": round_num(best_counter[row["attack_key"]] / samples),
                "top3_frequency": round_num(near_counter[row["attack_key"]] / samples),
            }
        )
    return output


def build_wide_matrix(rows: List[Dict[str, object]], attacks: List[AttackState], defenses: List[DefenseState]) -> Tuple[List[str], List[List[
      object]]]:
    row_map = {(row["attack_key"], row["defense_key"]): float(row["total_loss"]) for row in rows}
    header = ["attack_key", "attack_name_cn"] + [defense.defense_key for defense in defenses]
    matrix_rows: List[List[object]] = []
    for attack in attacks:
        matrix_rows.append(
            [attack.action_key, attack.action_name_cn] + [round_num(row_map[(attack.action_key, defense.defense_key)]) for defense in defenses
      ]
        )
    return header, matrix_rows


def write_csv(path: Path, rows: List[Dict[str, object]], fieldnames: List[str]) -> None:
    with path.open("w", encoding="utf-8-sig", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)


def write_wide_csv(path: Path, header: List[str], rows: List[List[object]]) -> None:
```

问题三核心代码

```python
#!/usr/bin/env python3
"""Solve MathorCup B Question 3 with PM01 Q1/Q2 results.

The model is a finite-horizon robust Markov game for a single 300 s match.
It consumes the Q1 attack ranking and Q2 attack-defense loss matrix produced
in this workspace, then exports policy tables, an opening battle plan, and
paper-ready figures.
"""

from __future__ import annotations

import csv
import json
import math
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, Iterable, List, Tuple

import matplotlib
import numpy as np

matplotlib.use("Agg")
from matplotlib import pyplot as plt
from matplotlib.colors import ListedColormap

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
```

```python
PAPER_BG = "#ffffff"
GRID_COLOR = "#d8ddd6"
NAVY = "#274c5e"
TEAL = "#4f8f8a"
SAGE = "#7b9e74"
AMBER = "#c99a46"
RUST = "#a85c4a"
SLATE = "#7a8793"
ACTION_PALETTE = [NAVY, TEAL, SAGE, AMBER, RUST, SLATE, "#b7a27a", "#5e6f73", "#8d6f64"]
Q3_GROUP_COLORS = {
    "fight": NAVY,
    "pause": AMBER,
    "reset": SAGE,
    "repair": RUST,
    "idle": SLATE,
}
Q3_HEATMAP_FIXED_COLORS = {
    "straight_punch": NAVY,
    "charge": AMBER,
    "head_guard_stance": SAGE,
    "idle": RUST,
}
Q3_HEATMAP_FALLBACK = [TEAL, "#b78a52", "#90ab7c", "#9a6458", "#7f8c99", "#5e6f73"]


WORKSPACE_ROOT = Path(__file__).resolve().parents[1]
Q1_RANKING_JSON = WORKSPACE_ROOT / "outputs" / "q1_complete_solution" / "q1_priority_ranking.json"
Q1_PORTRAITS_JSON = WORKSPACE_ROOT / "data" / "question1_action_portraits" / "q1_action_portraits_pm01.json"
Q2_LOSS_CSV = WORKSPACE_ROOT / "outputs" / "q2_complete_solution" / "q2_loss_matrix_long.csv"
Q2_DEFENSE_JSON = WORKSPACE_ROOT / "data" / "question2_defense_portraits" / "q2_defense_portraits_pm01.json"
OUTPUT_DIR = WORKSPACE_ROOT / "outputs" / "q3_complete_solution"
DOCS_FIGURES_DIR = WORKSPACE_ROOT / "docs" / "MathorCup论文模板-LaTeX" / "MathorCupmodeling-1.0" / "figures"

STEPS = 15
GAMMA = 0.955
ROBUST_COEFF = 0.72
TOTAL_TIME_S = 300.0
ATTACK_URGENCY = 0.34

H_LEVELS = [0.0, 0.25, 0.50, 0.75, 1.00]
B_LEVELS = [0.0, 0.33, 0.66, 1.00]
DIST_LABEL_CN = {0: "近距", 1: "中距", 2: "远距"}
DIST_LABEL_EN = {0: "near", 1: "mid", 2: "far"}


@dataclass(frozen=True)
class UnifiedAction:
    action_key: str
    action_name_cn: str
    kind: str
    style: str
    score_value: float
    damage_value: float
    stability_reserve: float
    balance_risk: float
    energy_cost: float
    duration_s: float
    recovery_s: float
    transition_s: float
    range_type: str
    tactical_availability: float


@dataclass(frozen=True)
class PairMetrics:
    damage_out: float
    damage_in: float
    balance_out_delta: float
    balance_in_delta: float
    reward: float


State = Tuple[int, int, int, int, int]  # h_self, h_opp, b_self, b_opp, distance


def q3_action_group(action: UnifiedAction) -> str:
    if action.kind == "attack":
        return "fight"
    if action.action_key in {"impact_buffer", "step_adjust", "com_compensation"}:
        return "pause"
    if action.action_key in {"controlled_fall", "grounded_guard", "quick_getup"}:
        return "repair"
    if action.kind == "defense":
        return "reset"
    return "idle"


def q3_action_color(action: UnifiedAction) -> str:
    return Q3_GROUP_COLORS[q3_action_group(action)]
```

```python
def hex_is_dark(color: str) -> bool:
    color = color.lstrip("#")
    r = int(color[0:2], 16)
    g = int(color[2:4], 16)
    b = int(color[4:6], 16)
    luminance = 0.299 * r + 0.587 * g + 0.114 * b
    return luminance < 150


def clamp(value: float, lower: float, upper: float) -> float:
    return max(lower, min(upper, value))


def round_num(value: float, digits: int = 6) -> float:
    return round(float(value), digits)


def nearest_index(levels: List[float], value: float) -> int:
    distances = [abs(item - value) for item in levels]
    return int(min(range(len(levels)), key=lambda idx: distances[idx]))


def load_json(path: Path) -> Dict[str, object]:
    return json.loads(path.read_text(encoding="utf-8"))


def attack_range_type(action_key: str, category: str) -> str:
    near = {"straight_punch", "hook_punch", "swing_punch", "combo_punch", "low_sweep", "knee_strike"}
    mid = {"front_kick", "side_kick", "spin_kick", "punch_kick_combo", "five_kick_combo"}
    if action_key in near:
        return "near"
    if action_key in mid:
        return "mid"
    if action_key == "charge":
        return "far"
    if action_key == "ground_counter":
        return "ground"
    return "mid" if category == "kick" else "near"


def load_attacks() -> Dict[str, UnifiedAction]:
    q1_payload = load_json(Q1_RANKING_JSON)
    portrait_payload = load_json(Q1_PORTRAITS_JSON)
    portrait_map = {
        row["pm01_calibrated_profile"]["action_key"]: row["pm01_calibrated_profile"]
        for row in portrait_payload["actions"]
    }

    ranking = q1_payload["ranking"]
    max_q = max(float(row["q_star"]) for row in ranking)
    max_strike = max(float(row["strike_best_site"]) for row in ranking)
    max_energy = max(float(row["energy_cost"]) for row in ranking)

    attacks: Dict[str, UnifiedAction] = {}
    for row in ranking:
        key = str(row["action_key"])
        portrait = portrait_map[key]
        q_norm = float(row["q_star"]) / max_q if max_q else 0.0
        strike_norm = float(row["strike_best_site"]) / max_strike if max_strike else 0.0
        energy_norm = float(row["energy_cost"]) / max_energy if max_energy else 0.0
        stability = float(row["stability_reserve"])
        support_penalty = float(row["single_support_ratio"])
        category = str(row["category"])
        damage = clamp(0.58 * max(q_norm, 0.0) + 0.42 * strike_norm, 0.04, 1.12)
        risk = clamp(
            0.62 * (1.0 - stability)
            + 0.18 * support_penalty
            + 0.12 * energy_norm
            + 0.08 * (1.0 - float(row["stability_score"])),
            0.02,
            0.95,
        )
        attacks[key] = UnifiedAction(
            action_key=key,
            action_name_cn=str(row["action_name_cn"]),
            kind="attack",
            style=category,
            score_value=float(row["q_star"]),
            damage_value=damage,
            stability_reserve=stability,
            balance_risk=risk,
            energy_cost=energy_norm,
            duration_s=float(portrait["duration_s"]),
            recovery_s=float(portrait["duration_s"]) * 0.35,
            transition_s=0.12 + 0.20 * support_penalty,
            range_type=attack_range_type(key, category),
            tactical_availability=float(row["tactical_availability"]),
        )
    return attacks
```

```python
def load_defenses() -> Dict[str, UnifiedAction]:
    payload = load_json(Q2_DEFENSE_JSON)
    defenses: Dict[str, UnifiedAction] = {}
    max_energy = max(float(row["mechanical_energy_j"]) for row in payload["defenses"])
    for row in payload["defenses"]:
        key = str(row["defense_key"])
        energy_norm = float(row["mechanical_energy_j"]) / max_energy if max_energy else 0.0
        balance_risk = clamp(
            0.45 * float(row["raw_stability_risk"])
            + 0.18 * float(row["single_support_ratio"])
            + 0.15 * energy_norm
            - 0.24 * float(row["balance_reserve_score"])
            - 0.12 * float(row["stability_score"]),
            0.015,
            0.90,
        )
        defenses[key] = UnifiedAction(
            action_key=key,
            action_name_cn=str(row["defense_name_cn"]),
            kind="defense",
            style=str(row["style"]),
            score_value=float(row["stability_score"]),
            damage_value=0.0,
            stability_reserve=float(row["balance_reserve_score"]),
            balance_risk=balance_risk,
            energy_cost=energy_norm,
            duration_s=float(row["duration_s"]),
            recovery_s=float(row["recovery_s"]),
            transition_s=float(row["transition_s"]),
            range_type="none",
            tactical_availability=float(row["timing_agility_score"]),
        )
    return defenses


def load_pair_matrix() -> Dict[Tuple[str, str], Dict[str, float]]:
    pairs: Dict[Tuple[str, str], Dict[str, float]] = {}
    with Q2_LOSS_CSV.open("r", encoding="utf-8-sig", newline="") as handle:
        reader = csv.DictReader(handle)
        for row in reader:
            key = (str(row["attack_key"]), str(row["defense_key"]))
            pairs[key] = {
                "intercept_score": float(row["intercept_score"]),
                "residual_damage": float(row["residual_damage"]),
                "balance_burden": float(row["balance_burden"]),
                "timing_penalty": float(row["timing_penalty"]),
                "sequence_penalty": float(row["sequence_penalty"]),
                "counter_bonus": float(row["counter_bonus"]),
                "total_loss": float(row["total_loss"]),
            }
    return pairs


def build_actions() -> Dict[str, UnifiedAction]:
    attacks = load_attacks()
    defenses = load_defenses()
    actions: Dict[str, UnifiedAction] = {}
    actions.update(attacks)
    actions.update(defenses)
    actions["idle"] = UnifiedAction(
        action_key="idle",
        action_name_cn="寰呗滿/瑋冩瞩",
        kind="idle",
        style="idle",
        score_value=0.0,
        damage_value=0.0,
        stability_reserve=0.72,
        balance_risk=-0.08,
        energy_cost=0.02,
        duration_s=0.55,
        recovery_s=0.16,
        transition_s=0.18,
        range_type="none",
        tactical_availability=0.70,
    )
    return actions


def range_factor(range_type: str, distance: int) -> float:
    table = {
        "near": [1.04, 0.58, 0.12],
        "mid": [0.70, 1.00, 0.52],
        "far": [0.38, 0.92, 1.10],
        "ground": [0.82, 0.35, 0.06],
        "none": [0.0, 0.0, 0.0],
    }
    return table.get(range_type, [0.6, 0.6, 0.6])[distance]


def movement_intent(action: UnifiedAction, balance_idx: int) -> int:
    if action.kind == "attack":
```

```
        if action.range_type == "far":
            return -2
        if action.range_type == "mid":
            return -1
        return 0
    if action.kind == "idle":
        return 0
    if action.action_key in {"backstep", "lateral_dodge", "sidestep_circle", "block_retreat_circle"}:
        return 1
    if action.action_key in {"step_adjust", "side_body_stance"} and balance_idx <= 1:
        return 0
    if action.action_key in {"impact_buffer", "com_compensation"}:
        return 1
    return 0


def is_terminal(state: State) -> bool:
    return state[0] == 0 or state[1] == 0 or state[2] >= 3 or state[3] >= 3


def is_action_feasible(action: UnifiedAction, balance_idx: int, distance: int) -> bool:
    unstable = balance_idx >= 2
    if action.kind == "attack":
        if action.action_key == "ground_counter":
            return unstable and distance == 0
        if unstable and action.action_key not in {"straight_punch", "hook_punch", "low_sweep"}:
            return False
        return range_factor(action.range_type, distance) >= 0.18

    if action.kind == "defense":
        if action.action_key in {"quick_getup", "grounded_guard", "controlled_fall"}:
            return unstable
        if unstable:
            return action.style in {"posture", "balance", "ground", "block"}
        if action.action_key in {"controlled_fall", "quick_getup"}:
            return False
        return True

    return True


def residual_multiplier(attack: UnifiedAction, defense: UnifiedAction, pairs: Dict[Tuple[str, str], Dict[str, float]]) -> float:
    if defense.kind != "defense":
        return 0.88 if defense.kind == "attack" else 1.00
    row = pairs.get((attack.action_key, defense.action_key))
    if not row:
        return 0.78
    intercept = row["intercept_score"]
    loss = row["total_loss"]
    multiplier = 0.08 + 0.82 * (1.0 - intercept) + 0.18 * loss
    return clamp(multiplier, 0.06, 0.95)


def defense_balance_bonus(action: UnifiedAction) -> float:
    if action.kind == "idle":
        return 0.09
    if action.kind != "defense":
        return 0.0
    bonus = 0.03
    if action.style == "posture":
        bonus += 0.08
    if action.style == "balance":
        bonus += 0.11
    if action.style == "ground":
        bonus += 0.06
    if action.action_key in {"side_body_stance", "step_adjust", "impact_buffer", "com_compensation", "low_center_stance"}:
        bonus += 0.05
    return bonus


def step_dynamics(state: State, our: UnifiedAction, opp: UnifiedAction, pairs: Dict[Tuple[str, str], Dict[str, float]]) -> Tuple[State,
      PairMetrics]:
    h_self = H_LEVELS[state[0]]
    h_opp = H_LEVELS[state[1]]
    b_self = B_LEVELS[state[2]]
    b_opp = B_LEVELS[state[3]]
    distance = state[4]

    damage_out = 0.0
    damage_in = 0.0

    if our.kind == "attack":
        damage_out = (
            our.damage_value
            * range_factor(our.range_type, distance)
            * residual_multiplier(our, opp, pairs)
            * (0.86 + 0.14 * our.tactical_availability)
        )
    if opp.kind == "attack":
        damage_in = (
            opp.damage_value
```

```
            * range_factor(opp.range_type, distance)
            * residual_multiplier(opp, our, pairs)
            * (0.86 + 0.14 * opp.tactical_availability)
        )

    if our.kind == "attack" and opp.kind == "attack":
        damage_out *= 0.88
        damage_in *= 0.88

    h_self_next = clamp(h_self - 0.30 * damage_in, 0.0, 1.0)
    h_opp_next = clamp(h_opp - 0.30 * damage_out, 0.0, 1.0)

    b_self_delta = (
        our.balance_risk
        + 0.42 * damage_in
        - defense_balance_bonus(our)
        - (0.02 if our.kind != "attack" else 0.0)
    )
    b_opp_delta = (
        opp.balance_risk
        + 0.42 * damage_out
        - defense_balance_bonus(opp)
        - (0.02 if opp.kind != "attack" else 0.0)
    )

    b_self_next = clamp(b_self + b_self_delta, 0.0, 1.0)
    b_opp_next = clamp(b_opp + b_opp_delta, 0.0, 1.0)

    dist_next = distance + movement_intent(our, state[2]) + movement_intent(opp, state[3])
    if our.kind == "attack" and opp.kind == "attack":
        dist_next -= 1
    if our.kind == "idle" and opp.kind == "idle":
        if dist_next > 1:
            dist_next -= 1
        elif dist_next < 1:
            dist_next += 1
    dist_next = int(clamp(float(dist_next), 0.0, 2.0))

    terminal_bonus = 0.0
    if h_opp_next <= 1e-8 or b_opp_next >= 1.0 - 1e-8:
        terminal_bonus += 2.2
    if h_self_next <= 1e-8 or b_self_next >= 1.0 - 1e-8:
        terminal_bonus -= 2.2

    own_balance_penalty = max(0.0, b_self_next - b_self)
    opp_balance_gain = max(0.0, b_opp_next - b_opp)
    energy_penalty = 0.10 * our.energy_cost + 0.025 * our.duration_s
    initiative_bonus = 0.05 if our.kind == "attack" else 0.0
    pressure_bonus = 0.22 * damage_out if our.kind == "attack" else 0.0
    defense_tempo_bonus = 0.0
    if our.kind == "defense" and opp.kind == "attack" and damage_in <= 0.08:
        defense_tempo_bonus = 0.10 + 0.04 * our.stability_reserve
    idle_penalty = 0.08 if our.kind == "idle" and b_self <= 0.33 and h_self >= h_opp else 0.0

    reward = (
        1.18 * (damage_out - damage_in)
        + 0.76 * opp_balance_gain
        - 1.05 * own_balance_penalty
        - energy_penalty
        + terminal_bonus
        + initiative_bonus
        + pressure_bonus
        + defense_tempo_bonus
        - idle_penalty
    )

    next_state = (
        nearest_index(H_LEVELS, h_self_next),
        nearest_index(H_LEVELS, h_opp_next),
        nearest_index(B_LEVELS, b_self_next),
        nearest_index(B_LEVELS, b_opp_next),
        dist_next,
    )
    return next_state, PairMetrics(
        damage_out=damage_out,
        damage_in=damage_in,
        balance_out_delta=b_opp_next - b_opp,
        balance_in_delta=b_self_next - b_self,
        reward=reward,
    )


def terminal_value(state: State) -> float:
    h_self = H_LEVELS[state[0]]
    h_opp = H_LEVELS[state[1]]
    b_self = B_LEVELS[state[2]]
    b_opp = B_LEVELS[state[3]]
    value = 2.6 * (h_self - h_opp) + 1.35 * (b_opp - b_self)
    if h_opp <= 1e-8 or b_opp >= 1.0 - 1e-8:
        value += 1.8
    if h_self <= 1e-8 or b_self >= 1.0 - 1e-8:
```

```
        value -= 1.8
    return value


def build_states() -> Tuple[List[State], Dict[State, int]]:
    states: List[State] = []
    for hs in range(len(H_LEVELS)):
        for ho in range(len(H_LEVELS)):
            for bs in range(len(B_LEVELS)):
                for bo in range(len(B_LEVELS)):
                    for distance in range(3):
                        states.append((hs, ho, bs, bo, distance))
    return states, {state: idx for idx, state in enumerate(states)}


def solve_game(actions: Dict[str, UnifiedAction], pairs: Dict[Tuple[str, str], Dict[str, float]]) -> Tuple[List[List[str]], List[List[str]],
      List[List[float]], List[State]]:
    states, state_to_id = build_states()
    n_states = len(states)
    action_keys = list(actions.keys())

    feasible: Dict[Tuple[int, int], List[str]] = {}
    for balance_idx in range(len(B_LEVELS)):
        for dist_idx in range(3):
            feasible[(balance_idx, dist_idx)] = [
                key for key in action_keys if is_action_feasible(actions[key], balance_idx, dist_idx)
            ]

    values: List[List[float]] = [[0.0 for _ in range(n_states)] for _ in range(STEPS + 1)]
    values[STEPS] = [terminal_value(state) for state in states]
    policy: List[List[str]] = [["terminal" for _ in range(n_states)] for _ in range(STEPS)]
    worst_response: List[List[str]] = [["terminal" for _ in range(n_states)] for _ in range(STEPS)]

    v_next = values[STEPS][:]
    transition_cache: Dict[Tuple[int, str, str], Tuple[int, PairMetrics]] = {}

    for t in range(STEPS - 1, -1, -1):
        v_curr = [0.0 for _ in range(n_states)]
        for sid, state in enumerate(states):
            if is_terminal(state):
                v_curr[sid] = terminal_value(state)
                continue

            our_candidates = feasible[(state[2], state[4])]
            opp_candidates = feasible[(state[3], state[4])]
            best_value = -1e18
            best_action = "idle"
            best_opp = "idle"

            for our_key in our_candidates:
                our_action = actions[our_key]
                opp_values: List[float] = []
                worst_value = 1e18
                local_worst = "idle"
                for opp_key in opp_candidates:
                    cache_key = (sid, our_key, opp_key)
                    if cache_key in transition_cache:
                        next_sid, metrics = transition_cache[cache_key]
                    else:
                        next_state, metrics = step_dynamics(state, our_action, actions[opp_key], pairs)
                        next_sid = state_to_id[next_state]
                        transition_cache[cache_key] = (next_sid, metrics)

                    value = metrics.reward + GAMMA * v_next[next_sid]
                    opp_values.append(value)
                    if value < worst_value:
                        worst_value = value
                        local_worst = opp_key

                avg_value = float(sum(opp_values) / max(len(opp_values), 1))
                robust_value = ROBUST_COEFF * worst_value + (1.0 - ROBUST_COEFF) * avg_value
                if our_action.kind == "attack":
                    robust_value += ATTACK_URGENCY * (t / max(STEPS - 1, 1))
                if our_action.kind == "defense" and state[2] >= 2:
                    robust_value += 0.16

                if robust_value > best_value:
                    best_value = robust_value
                    best_action = our_key
                    best_opp = local_worst

            v_curr[sid] = best_value
            policy[t][sid] = best_action
            worst_response[t][sid] = best_opp

        values[t] = v_curr
        v_next = v_curr

    return policy, worst_response, values, states
```

```python
def state_name(state: State) -> Dict[str, object]:
    return {
        "h_self": H_LEVELS[state[0]],
        "h_opp": H_LEVELS[state[1]],
        "b_self": B_LEVELS[state[2]],
        "b_opp": B_LEVELS[state[3]],
        "distance": DIST_LABEL_CN[state[4]],
    }


def write_policy_table(policy: List[List[str]], worst: List[List[str]], values: List[List[float]], states: List[State], actions: Dict[str,
      UnifiedAction]) -> Path:
    out = OUTPUT_DIR / "q3_policy_table.csv"
    with out.open("w", encoding="utf-8-sig", newline="") as handle:
        fieldnames = [
            "t_step",
            "remaining_time_s",
            "h_self",
            "h_opp",
            "b_self",
            "b_opp",
            "distance",
            "best_action_key",
            "best_action_cn",
            "best_action_kind",
            "worst_response_key",
            "worst_response_cn",
            "state_value",
        ]
        writer = csv.DictWriter(handle, fieldnames=fieldnames)
        writer.writeheader()
        dt = TOTAL_TIME_S / STEPS
        for t in range(STEPS):
            for sid, state in enumerate(states):
                best = policy[t][sid]
                opp = worst[t][sid]
                row = state_name(state)
                writer.writerow(
                    {
                        "t_step": t,
                        "remaining_time_s": round_num((STEPS - t) * dt, 3),
                        **row,
                        "best_action_key": best,
                        "best_action_cn": actions.get(best, actions["idle"]).action_name_cn,
                        "best_action_kind": actions.get(best, actions["idle"]).kind,
                        "worst_response_key": opp,
                        "worst_response_cn": actions.get(opp, actions["idle"]).action_name_cn,
                        "state_value": round_num(values[t][sid]),
                    }
                )
    return out


def simulate_opening_plan(policy: List[List[str]], worst: List[List[str]], values: List[List[float]], states: List[State], actions: Dict[str,
      UnifiedAction], pairs: Dict[Tuple[str, str], Dict[str, float]]) -> Tuple[List[Dict[str, object]], Dict[str, object]]:
    state_to_id = {state: idx for idx, state in enumerate(states)}
    state: State = (4, 4, 0, 0, 1)
    elapsed = 0.0
    rows: List[Dict[str, object]] = []
    cumulative_out = 0.0
    cumulative_in = 0.0

    for t in range(STEPS):
        if is_terminal(state):
            break
        sid = state_to_id[state]
        our_key = policy[t][sid]
        opp_key = worst[t][sid]
        our_action = actions[our_key]
        opp_action = actions[opp_key]
        next_state, metrics = step_dynamics(state, our_action, opp_action, pairs)
        cycle_s = clamp(
            max(our_action.duration_s, opp_action.duration_s)
            + 1.65
            + 0.18 * abs(next_state[4] - state[4])
            + 0.20 * (1 if our_action.kind == "attack" and opp_action.kind == "attack" else 0),
            1.35,
            6.50,
        )
        cumulative_out += metrics.damage_out
        cumulative_in += metrics.damage_in
        rows.append(
            {
                "step": t + 1,
                "time_s": round_num(elapsed, 3),
                "duration_s": round_num(cycle_s, 3),
                "distance": DIST_LABEL_CN[state[4]],
                "h_self": H_LEVELS[state[0]],
                "h_opp": H_LEVELS[state[1]],
                "b_self": B_LEVELS[state[2]],
                "b_opp": B_LEVELS[state[3]],
```

```
                "our_action_key": our_key,
                "our_action_cn": our_action.action_name_cn,
                "our_action_kind": our_action.kind,
                "worst_response_key": opp_key,
                "worst_response_cn": opp_action.action_name_cn,
                "damage_out": round_num(metrics.damage_out),
                "damage_in": round_num(metrics.damage_in),
                "balance_out_delta": round_num(metrics.balance_out_delta),
                "balance_in_delta": round_num(metrics.balance_in_delta),
                "reward": round_num(metrics.reward),
                "state_value": round_num(values[t][sid]),
                "cumulative_damage_out": round_num(cumulative_out),
                "cumulative_damage_in": round_num(cumulative_in),
            }
        )
        elapsed += cycle_s
        state = next_state

    out = OUTPUT_DIR / "q3_opening_battle_plan.csv"
    with out.open("w", encoding="utf-8-sig", newline="") as handle:
        fieldnames = list(rows[0].keys()) if rows else []
        writer = csv.DictWriter(handle, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(rows)

    final = state_name(state)
    summary = {
        "initial_state": {"h_self": 1.0, "h_opp": 1.0, "b_self": 0.0, "b_opp": 0.0, "distance": "中距"},
        "final_state_under_worst_response": final,
        "steps": len(rows),
        "elapsed_s": round_num(elapsed, 3),
        "cumulative_damage_out": round_num(cumulative_out),
        "cumulative_damage_in": round_num(cumulative_in),
        "net_damage": round_num(cumulative_out - cumulative_in),
        "final_value": round_num(values[min(len(rows), STEPS)][state_to_id[state]] if len(rows) < STEPS else terminal_value(state)),
        "opening_plan_path": str(out.relative_to(WORKSPACE_ROOT)),
    }
    return rows, summary


def action_distribution(policy: List[List[str]], states: List[State], actions: Dict[str, UnifiedAction], t: int = 0) -> List[Dict[str, object
    ]]:
    counter: Dict[str, int] = {}
    non_terminal_count = 0
    for sid, state in enumerate(states):
        if is_terminal(state):
            continue
        non_terminal_count += 1
        key = policy[t][sid]
        counter[key] = counter.get(key, 0) + 1
    rows = []
    for key, count in sorted(counter.items(), key=lambda item: item[1], reverse=True):
        rows.append(
            {
                "action_key": key,
                "action_cn": actions[key].action_name_cn,
                "kind": actions[key].kind,
                "count": count,
                "ratio": round_num(count / max(non_terminal_count, 1), 4),
            }
        )
    return rows


def write_distribution_csv(rows: List[Dict[str, object]]) -> Path:
    out = OUTPUT_DIR / "q3_opening_action_distribution.csv"
    with out.open("w", encoding="utf-8-sig", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=["action_key", "action_cn", "kind", "count", "ratio"])
        writer.writeheader()
        writer.writerows(rows)
    return out


def rule_summary(policy: List[List[str]], states: List[State], actions: Dict[str, UnifiedAction]) -> List[Dict[str, object]]:
    buckets: Dict[Tuple[str, str, float], Dict[str, int]] = {}
    for sid, state in enumerate(states):
        if is_terminal(state):
            continue
        h_self = H_LEVELS[state[0]]
        h_opp = H_LEVELS[state[1]]
        if h_self > h_opp:
            health_rel = "领先"
        elif h_self < h_opp:
            health_rel = "落后"
        else:
            health_rel = "均势"
        key = (health_rel, DIST_LABEL_CN[state[4]], B_LEVELS[state[2]])
        action = policy[0][sid]
        buckets.setdefault(key, {})
        buckets[key][action] = buckets[key].get(action, 0) + 1
```

```python
    rows: List[Dict[str, object]] = []
    for key, counter in sorted(buckets.items(), key=lambda item: (item[0][0], item[0][1], item[0][2])):
        total = sum(counter.values())
        top = sorted(counter.items(), key=lambda item: item[1], reverse=True)[:3]
        rows.append(
            {
                "health_relation": key[0],
                "distance": key[1],
                "b_self": key[2],
                "top_actions": "、".join(actions[name].action_name_cn for name, _ in top),
                "top_ratios": "、".join(f"{count / max(total, 1):.2f}" for _, count in top),
                "support_count": total,
            }
        )
    out = OUTPUT_DIR / "q3_rule_summary.csv"
    with out.open("w", encoding="utf-8-sig", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=["health_relation", "distance", "b_self", "top_actions", "top_ratios", "support_count"])
        writer.writeheader()
        writer.writerows(rows)
    return rows
```

问题四核心代码

```python
#!/usr/bin/env python3
"""Solve MathorCup B Question 4 with a BO3 resource-decision DP.

The model combines:
- Q1 attack quality, Q2 defense robustness, and Q3 single-match strategy value;
- BO3 score state and resource constraints;
- in-game advantage, fault level, energy, and balance risk;
- dynamic programming over resource actions: fight, tactical pause, manual reset,
  and emergency repair.
"""

from __future__ import annotations

import csv
import json
import math
import shutil
from collections import defaultdict
from dataclasses import dataclass
from functools import lru_cache
from pathlib import Path
from typing import Dict, Iterable, List, Tuple

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import ListedColormap


WORKSPACE_ROOT = Path(__file__).resolve().parents[1]
OUTPUT_DIR = WORKSPACE_ROOT / "outputs" / "q4_complete_solution"
FIGURES_DIR = WORKSPACE_ROOT / "docs" / "MathorCup论文模板-LaTeX" / "MathorCupmodeling-1.0" / "figures"

Q1_RANKING_JSON = WORKSPACE_ROOT / "outputs" / "q1_complete_solution" / "q1_priority_ranking.json"
Q2_SUMMARY_JSON = WORKSPACE_ROOT / "outputs" / "q2_complete_solution" / "q2_summary.json"
Q3_SUMMARY_JSON = WORKSPACE_ROOT / "outputs" / "q3_complete_solution" / "q3_strategy_summary.json"

N_STEPS = 12
GAME_TIME_S = 300.0
STEP_TIME_S = GAME_TIME_S / N_STEPS

ADV_LEVELS = [-2, -1, 0, 1, 2]
ADV_CN = {
    -2: "明显落后",
    -1: "略微落后",
    0: "均势",
    1: "略微领先",
    2: "明显领先",
}
FAULT_CN = {
    0: "无故障",
    1: "轻微故障",
    2: "中等故障",
    3: "严重故障",
}
ENERGY_CN = {
    0: "低能量",
    1: "中能量",
    2: "高能量",
}
BALANCE_CN = {
    0: "稳定",
    1: "预警",
    2: "高风险",
    3: "倒地/临界",
}
```

```python
ACTION_CN = {
    "fight": "常规攻防",
    "pause": "战术暂停",
    "reset": "人工复位",
    "repair": "紧急维修",
}

ACTION_ORDER = ["fight", "pause", "reset", "repair"]
ACTION_COLOR = {
    "fight": "#274c5e",
    "pause": "#c99a46",
    "reset": "#7b9e74",
    "repair": "#a85c4a",
}
PAPER_BG = "#ffffff"
GRID_COLOR = "#d8ddd6"
SLATE = "#7a8793"

State = Tuple[int, int, int, int, int, int, int, int, int, int]
# w, l, k, advantage, fault, energy, balance, pause_left, reset_left, repair_left


@dataclass(frozen=True)
class TacticalInputs:
    attack_quality: float
    defense_quality: float
    q3_edge: float
    q3_initial_value: float
    q3_winning_ratio: float
    q3_pressure_value: float


def clamp(value: float, low: float, high: float) -> float:
    return max(low, min(high, value))


def sigmoid(value: float) -> float:
    return 1.0 / (1.0 + math.exp(-value))


def load_json(path: Path) -> Dict[str, object]:
    with path.open("r", encoding="utf-8") as handle:
        return json.load(handle)


def setup_matplotlib() -> None:
    plt.rcParams["font.sans-serif"] = [
        "Microsoft YaHei",
        "SimHei",
        "SimSun",
        "Noto Sans CJK SC",
        "Arial Unicode MS",
    ]
    plt.rcParams["axes.unicode_minus"] = False


def scenario_key(w: int, l: int) -> str:
    if w == 1 and l == 0:
        return "lead"
    if w == 0 and l == 1:
        return "behind"
    if w == 1 and l == 1:
        return "decider"
    return "tie"


def scenario_cn(key: str) -> str:
    return {
        "tie": "开局均势(0:0)",
        "lead": "领先(1:0)",
        "behind": "落后(0:1)",
        "decider": "决胜局(1:1)",
    }[key]


def stage_cn(k: int) -> str:
    ratio = k / max(N_STEPS - 1, 1)
    if ratio < 0.34:
        return "开局"
    if ratio < 0.67:
        return "中局"
    return "终局"


def clamp_adv(value: int) -> int:
    return int(clamp(value, ADV_LEVELS[0], ADV_LEVELS[-1]))


def load_tactical_inputs() -> TacticalInputs:
    q1 = load_json(Q1_RANKING_JSON)
```

```
    q2 = load_json(Q2_SUMMARY_JSON)
    q3 = load_json(Q3_SUMMARY_JSON)

    ranking = list(q1["ranking"])
    max_q = max(float(row["q_star"]) for row in ranking)
    top5 = sorted((float(row["q_star"]) / max_q for row in ranking), reverse=True)[:5]
    attack_quality = float(sum(top5) / len(top5))

    top_line = list(q2["top_line"])
    mean_best_loss = sum(float(row["best_loss"]) for row in top_line) / len(top_line)
    mean_near_opt = sum(float(row["near_opt_count"]) for row in top_line) / len(top_line)
    defense_quality = clamp(1.0 - 1.65 * mean_best_loss + 0.018 * mean_near_opt, 0.0, 1.0)

    initial_value = float(q3["initial_state_value"])
    winning_ratio = float(q3["winning_tendency_ratio"])
    pressure = float(q3["opening_plan_summary"]["final_value"])
    net_damage = float(q3["opening_plan_summary"]["net_damage"])
    q3_edge = (
        0.060 * clamp(initial_value / 2.2, -1.0, 1.0)
        + 0.130 * (winning_ratio - 0.5)
        + 0.018 * pressure
        + 0.018 * clamp(net_damage, -2.0, 2.0)
        + 0.035 * (attack_quality - 0.75)
        + 0.030 * (defense_quality - 0.75)
    )
    q3_edge = clamp(q3_edge, -0.10, 0.16)

    return TacticalInputs(
        attack_quality=attack_quality,
        defense_quality=defense_quality,
        q3_edge=q3_edge,
        q3_initial_value=initial_value,
        q3_winning_ratio=winning_ratio,
        q3_pressure_value=pressure,
    )


def game_start_state(w: int, l: int, pause: int, reset: int, repair: int, carry_fault: int = 0) -> State:
    # Between games the robot receives routine inspection: balance and energy are restored,
    # while unresolved structural faults only drop one level.
    return (
        w,
        l,
        0,
        0,
        max(0, carry_fault - 1),
        2,
        0,
        pause,
        reset,
        repair,
    )


class ResourceDP:
    def __init__(self, tactical: TacticalInputs) -> None:
        self.tactical = tactical
        self.policy: Dict[State, str] = {}
        self.q_values: Dict[State, Dict[str, float]] = {}

    def combat_edge(self, state: State) -> float:
        w, l, k, adv, fault, energy, balance, *_ = state
        scene = scenario_key(w, l)
        scene_edge = {
            "tie": 0.000,
            "lead": -0.030,
            "behind": 0.045,
            "decider": 0.025,
        }[scene]
        stage_pressure = 0.015 if k >= 8 and scene in {"behind", "decider"} else 0.0
        return (
            self.tactical.q3_edge
            + scene_edge
            + stage_pressure
            + 0.120 * adv
            + 0.065 * (energy - 1)
            - [0.0, 0.105, 0.255, 0.500][fault]
            - [0.0, 0.080, 0.215, 0.390][balance]
        )

    def fault_probability(self, state: State) -> float:
        w, l, k, adv, fault, energy, balance, *_ = state
        scene = scenario_key(w, l)
        stage_base = 0.050 + 0.018 * min(2, k // 4)
        scene_stress = {
            "tie": 0.000,
            "lead": -0.010,
            "behind": 0.035,
            "decider": 0.030,
        }[scene]
        value = (
```

```
        stage_base
        + scene_stress
        + 0.034 * balance
        + 0.030 * max(0, 1 - energy)
        + 0.040 * fault
        + (0.020 if adv < 0 else 0.0)
    )
    return clamp(value, 0.015, 0.430)

def forced_loss_probability(self, state: State) -> float:
    _, _, k, adv, fault, energy, balance, *_ = state
    value = 0.0
    if fault >= 2 or balance >= 2:
        value += 0.030 + 0.055 * fault + 0.050 * balance
    if fault == 3:
        value += 0.080
    if balance == 3:
        value += 0.070
    value -= 0.030 * energy
    value += 0.020 if k >= 8 else 0.0
    value += 0.015 if adv < 0 else 0.0
    return clamp(value, 0.0, 0.520)

def game_win_probability(self, state: State) -> float:
    w, l, _, adv, fault, energy, balance, *_ = state
    scene = scenario_key(w, l)
    scene_finish = {
        "tie": 0.000,
        "lead": -0.020,
        "behind": 0.025,
        "decider": 0.010,
    }[scene]
    logit = (
        self.tactical.q3_edge
        + scene_finish
        + 0.950 * adv
        + 0.160 * (energy - 1)
        - 0.360 * balance
        - 0.470 * fault
    )
    return clamp(sigmoid(logit), 0.025, 0.975)

def round_result_state(self, state: State, win: bool) -> State:
    w, l, _, _, fault, _, _, pause, reset, repair = state
    if win:
        return game_start_state(w + 1, l, pause, reset, repair, fault)
    return game_start_state(w, l + 1, pause, reset, repair, fault)

def round_end_transitions(self, state: State) -> List[Tuple[float, State]]:
    p_win = self.game_win_probability(state)
    return [
        (p_win, self.round_result_state(state, True)),
        (1.0 - p_win, self.round_result_state(state, False)),
    ]

def fight_transitions(self, state: State) -> List[Tuple[float, State]]:
    w, l, k, adv, fault, energy, balance, pause, reset, repair = state
    p_forced_loss = self.forced_loss_probability(state)
    transitions: List[Tuple[float, State]] = []
    if p_forced_loss > 0:
        transitions.append((p_forced_loss, self.round_result_state(state, False)))

    edge = self.combat_edge(state)
    score_signal = math.tanh(1.55 * edge)
    p_up = 0.285 + 0.170 * score_signal - 0.020 * fault - 0.015 * balance + 0.014 * (energy - 1)
    p_down = 0.270 - 0.165 * score_signal + 0.032 * fault + 0.022 * balance - 0.010 * (energy - 1)
    p_up = clamp(p_up, 0.045, 0.600)
    p_down = clamp(p_down, 0.045, 0.650)
    if p_up + p_down > 0.900:
        scale = 0.900 / (p_up + p_down)
        p_up *= scale
        p_down *= scale
    score_cases = [
        (p_down, -1),
        (1.0 - p_up - p_down, 0),
        (p_up, 1),
    ]

    p_fault = self.fault_probability(state)
    p_normal_mass = 1.0 - p_forced_loss
    for p_score, delta in score_cases:
        for p_f, fault_inc in [(1.0 - p_fault, 0), (p_fault, 1)]:
            prob = p_normal_mass * p_score * p_f
            if prob <= 0:
                continue
            next_fault = min(3, fault + fault_inc)
            next_adv = clamp_adv(adv + delta)

            fatigue_drop = 1 if (k in {3, 6, 9} or fault >= 2 or balance >= 2) else 0
            next_energy = int(clamp(energy - fatigue_drop, 0, 2))
```

```
            bal_change = 0
            if delta < 0:
                bal_change += 1
            if fault_inc and fault >= 1:
                bal_change += 1
            if energy == 0 and delta <= 0:
                bal_change += 1
            if delta > 0 and balance > 0:
                bal_change -= 1
            next_balance = int(clamp(balance + bal_change, 0, 3))
            transitions.append(
                (
                    prob,
                    (
                        w,
                        l,
                        min(N_STEPS, k + 1),
                        next_adv,
                        next_fault,
                        next_energy,
                        next_balance,
                        pause,
                        reset,
                        repair,
                    ),
                )
            )
    return merge_transitions(transitions)

def pause_transitions(self, state: State) -> List[Tuple[float, State]]:
    w, l, k, adv, fault, energy, balance, pause, reset, repair = state
    if pause <= 0:
        return []
    scene = scenario_key(w, l)
    p_improve = 0.220
    if adv <= 0:
        p_improve += 0.105
    if scene in {"behind", "decider"}:
        p_improve += 0.045
    if fault >= 2:
        p_improve -= 0.060
    p_improve = clamp(p_improve, 0.080, 0.440)

    next_common = (
        w,
        l,
        min(N_STEPS, k + 1),
        adv,
        fault,
        min(2, energy + 1),
        max(0, balance - 1),
        pause - 1,
        reset,
        repair,
    )
    improved = list(next_common)
    improved[3] = clamp_adv(adv + 1)
    return merge_transitions([(1.0 - p_improve, next_common), (p_improve, tuple(improved))])

def reset_transitions(self, state: State) -> List[Tuple[float, State]]:
    w, l, k, adv, fault, energy, balance, pause, reset, repair = state
    if reset <= 0:
        return []
    base_adv = clamp_adv(adv - 1)
    saved_adv = adv if balance >= 3 or fault >= 2 else base_adv
    next_fault = max(0, fault - 1)
    next_common = (
        w,
        l,
        min(N_STEPS, k + 1),
        base_adv,
        next_fault,
        max(1, energy),
        0,
        pause,
        reset - 1,
        repair,
    )
    saved = list(next_common)
    saved[3] = saved_adv
    p_saved = 0.280 + 0.100 * (1 if balance >= 2 else 0) + 0.080 * (1 if fault >= 2 else 0)
    p_saved = clamp(p_saved, 0.160, 0.520)
    return merge_transitions([(1.0 - p_saved, next_common), (p_saved, tuple(saved))])

def repair_transitions(self, state: State) -> List[Tuple[float, State]]:
    w, l, k, adv, fault, energy, balance, pause, reset, repair = state
    if repair <= 0:
        return []
    # Emergency repair is legal even before the fault becomes severe, but it has
    # a tempo cost. DP will only select it when the future value compensates.
    tempo_penalty = 2 if fault <= 1 and balance <= 1 else 1
```

```
        next_adv = clamp_adv(adv - tempo_penalty)
        p_stabilize = 0.760 if fault >= 2 or balance >= 2 else 0.580
        repaired = (
            w,
            l,
            min(N_STEPS, k + 2),
            next_adv,
            0,
            2,
            0,
            pause,
            reset,
            repair - 1,
        )
        partial = (
            w,
            l,
            min(N_STEPS, k + 2),
            next_adv,
            max(0, fault - 2),
            2,
            max(0, balance - 2),
            pause,
            reset,
            repair - 1,
        )
        return merge_transitions([(p_stabilize, repaired), (1.0 - p_stabilize, partial)])

    def feasible_actions(self, state: State) -> List[str]:
        _, _, _, _, fault, _, balance, pause, reset, repair = state
        actions = ["fight"]
        if pause > 0:
            actions.append("pause")
        if reset > 0 and (balance >= 1 or fault >= 1):
            actions.append("reset")
        if repair > 0 and (fault >= 1 or balance >= 3):
            actions.append("repair")
        return actions

    def action_transitions(self, state: State, action: str) -> List[Tuple[float, State]]:
        if action == "fight":
            return self.fight_transitions(state)
        if action == "pause":
            return self.pause_transitions(state)
        if action == "reset":
            return self.reset_transitions(state)
        if action == "repair":
            return self.repair_transitions(state)
        raise ValueError(action)

    @lru_cache(maxsize=None)
    def value(self, state: State) -> float:
        w, l, k, *_ = state
        if w >= 2:
            return 1.0
        if l >= 2:
            return 0.0
        if k >= N_STEPS:
            return sum(prob * self.value(next_state) for prob, next_state in self.round_end_transitions(state))

        qvals: Dict[str, float] = {}
        for action in self.feasible_actions(state):
            transitions = self.action_transitions(state, action)
            if not transitions:
                continue
            qvals[action] = sum(prob * self.value(next_state) for prob, next_state in transitions)

        best_action = max(qvals, key=lambda name: (qvals[name], -ACTION_ORDER.index(name)))
        self.policy[state] = best_action
        self.q_values[state] = qvals
        return qvals[best_action]


def merge_transitions(transitions: Iterable[Tuple[float, State]]) -> List[Tuple[float, State]]:
    merged: Dict[State, float] = defaultdict(float)
    for prob, state in transitions:
        if prob > 1e-12:
            merged[state] += prob
    total = sum(merged.values())
    if total <= 0:
        return []
    return [(prob / total, state) for state, prob in merged.items()]


def simulate_policy_distribution(dp: ResourceDP, initial_state: State) -> Dict[str, object]:
    distribution: Dict[State, float] = {initial_state: 1.0}
    expected_usage = {"pause": 0.0, "reset": 0.0, "repair": 0.0}
    first_usage_mass = {"pause": 0.0, "reset": 0.0, "repair": 0.0}
    first_usage_time = {"pause": 0.0, "reset": 0.0, "repair": 0.0}
    series_win_prob = 0.0
    series_loss_prob = 0.0
```

```
    max_iter = 96

    # Track paths by adding used-resource flags to the distribution key.
    dist_flags: Dict[Tuple[State, int, int, int], float] = {(initial_state, 0, 0, 0): 1.0}
    for _ in range(max_iter):
        if not dist_flags:
            break
        next_dist: Dict[Tuple[State, int, int, int], float] = defaultdict(float)
        for (state, used_p, used_r, used_m), mass in dist_flags.items():
            w, l, k, *_ = state
            if w >= 2:
                series_win_prob += mass
                continue
            if l >= 2:
                series_loss_prob += mass
                continue
            if k >= N_STEPS:
                transitions = dp.round_end_transitions(state)
                action = "round_end"
            else:
                action = dp.policy.get(state, "fight")
                transitions = dp.action_transitions(state, action)

            for prob, next_state in transitions:
                next_mass = mass * prob
                n_used_p, n_used_r, n_used_m = used_p, used_r, used_m
                if action == "pause":
                    expected_usage["pause"] += next_mass
                    if not used_p:
                        first_usage_mass["pause"] += next_mass
                        first_usage_time["pause"] += next_mass * (k * STEP_TIME_S)
                    n_used_p = 1
                elif action == "reset":
                    expected_usage["reset"] += next_mass
                    if not used_r:
                        first_usage_mass["reset"] += next_mass
                        first_usage_time["reset"] += next_mass * (k * STEP_TIME_S)
                    n_used_r = 1
                elif action == "repair":
                    expected_usage["repair"] += next_mass
                    if not used_m:
                        first_usage_mass["repair"] += next_mass
                        first_usage_time["repair"] += next_mass * (k * STEP_TIME_S)
                    n_used_m = 1
                next_dist[(next_state, n_used_p, n_used_r, n_used_m)] += next_mass
        dist_flags = dict(next_dist)

    first_usage_avg = {}
    for key in first_usage_mass:
        if first_usage_mass[key] > 1e-12:
            first_usage_avg[key] = first_usage_time[key] / first_usage_mass[key]
        else:
            first_usage_avg[key] = None

    return {
        "series_win_prob": series_win_prob,
        "series_loss_prob": series_loss_prob,
        "unresolved_mass": sum(dist_flags.values()),
        "expected_usage": expected_usage,
        "first_usage_time_s": first_usage_avg,
    }


def canonical_rows(dp: ResourceDP) -> List[Dict[str, object]]:
    rows: List[Dict[str, object]] = []
    score_states = [(0, 0), (1, 0), (0, 1), (1, 1)]
    for w, l in score_states:
        for adv in ADV_LEVELS:
            for fault in range(4):
                for energy in range(3):
                    for balance in range(4):
                        state = (w, l, 6, adv, fault, energy, balance, 2, 2, 1)
                        value = dp.value(state)
                        action = dp.policy.get(state, "fight")
                        rows.append(
                            {
                                "score": f"{w}:{l}",
                                "scenario": scenario_cn(scenario_key(w, l)),
                                "stage": stage_cn(6),
                                "advantage": adv,
                                "advantage_cn": ADV_CN[adv],
                                "fault": fault,
                                "fault_cn": FAULT_CN[fault],
                                "energy": energy,
                                "energy_cn": ENERGY_CN[energy],
                                "balance": balance,
                                "balance_cn": BALANCE_CN[balance],
                                "best_action": action,
                                "best_action_cn": ACTION_CN[action],
                                "series_value": round(value, 6),
                            }
```

```
                        )
    return rows


def key_state_rows(dp: ResourceDP) -> List[Dict[str, object]]:
    specs = [
        (0, 0, 0, 0, 0, 2, 0, 2, 2, 1, "满资源开局"),
        (1, 0, 0, 0, 0, 2, 0, 2, 2, 1, "1:0领先满资源"),
        (0, 1, 0, 0, 0, 2, 0, 2, 2, 1, "0:1落后满资源"),
        (1, 1, 0, 0, 0, 2, 0, 2, 2, 1, "1:1决胜满资源"),
        (0, 1, 8, -1, 1, 1, 1, 1, 1, 1, "落后局中后段小劣"),
        (1, 0, 8, 1, 1, 1, 1, 1, 2, 1, "领先局中后段小优"),
        (1, 1, 9, -1, 2, 0, 2, 1, 1, 1, "决胜局高压故障"),
        (0, 0, 7, 0, 3, 1, 2, 2, 2, 1, "首局严重故障"),
    ]
    rows: List[Dict[str, object]] = []
    for w, l, k, adv, fault, energy, balance, pause, reset, repair, name in specs:
        state = (w, l, k, adv, fault, energy, balance, pause, reset, repair)
        value = dp.value(state)
        action = dp.policy.get(state, "fight")
        qvals = dp.q_values.get(state, {})
        rows.append(
            {
                "scene_name": name,
                "score": f"{w}:{l}",
                "scenario": scenario_cn(scenario_key(w, l)),
                "stage": stage_cn(k),
                "time_s": round(k * STEP_TIME_S, 1),
                "advantage_cn": ADV_CN[adv],
                "fault_cn": FAULT_CN[fault],
                "energy_cn": ENERGY_CN[energy],
                "balance_cn": BALANCE_CN[balance],
                "resources": f"暂停{pause}/复位{reset}/维修{repair}",
                "best_action": action,
                "best_action_cn": ACTION_CN[action],
                "series_value": round(value, 6),
                "fight_value": round(qvals.get("fight", float("nan")), 6) if "fight" in qvals else "",
                "pause_value": round(qvals.get("pause", float("nan")), 6) if "pause" in qvals else "",
                "reset_value": round(qvals.get("reset", float("nan")), 6) if "reset" in qvals else "",
                "repair_value": round(qvals.get("repair", float("nan")), 6) if "repair" in qvals else "",
            }
        )
    return rows


def scenario_policy_rows(dp: ResourceDP) -> List[Dict[str, object]]:
    rows: List[Dict[str, object]] = []
    for w, l in [(0, 0), (1, 0), (0, 1), (1, 1)]:
        scene = scenario_key(w, l)
        for adv in [-1, 0, 1]:
            for fault in range(4):
                # Use representative mid-game energy/balance. Higher fault implies at
                # least warning-level balance risk.
                balance = 0 if fault == 0 else min(3, fault)
                energy = 1 if fault < 2 else 0
                state = (w, l, 6, adv, fault, energy, balance, 2, 2, 1)
                value = dp.value(state)
                action = dp.policy.get(state, "fight")
                rows.append(
                    {
                        "scenario": scenario_cn(scene),
                        "score": f"{w}:{l}",
                        "in_game_advantage": ADV_CN[adv],
                        "fault": FAULT_CN[fault],
                        "balance": BALANCE_CN[balance],
                        "energy": ENERGY_CN[energy],
                        "best_action_cn": ACTION_CN[action],
                        "best_action": action,
                        "series_value": round(value, 6),
                    }
                )
    return rows


def resource_marginal_rows(dp: ResourceDP) -> List[Dict[str, object]]:
    baseline = (0, 0, 0, 0, 0, 2, 0, 2, 2, 1)
    full_value = dp.value(baseline)
    rows = []
    variants = [
        ("暂停次数", "p", (0, 0, 0, 0, 0, 2, 0, 0, 2, 1), (0, 0, 0, 0, 0, 2, 0, 2, 2, 1)),
        ("复位次数", "r", (0, 0, 0, 0, 0, 2, 0, 2, 0, 1), (0, 0, 0, 0, 0, 2, 0, 2, 2, 1)),
        ("维修次数", "m", (0, 0, 0, 0, 0, 2, 0, 2, 2, 0), (0, 0, 0, 0, 0, 2, 0, 2, 2, 1)),
    ]
    for name, code, low_state, high_state in variants:
        low_value = dp.value(low_state)
        high_value = dp.value(high_state)
        rows.append(
            {
                "resource": name,
                "code": code,
                "low_value": round(low_value, 6),
```

```
                "high_value": round(high_value, 6),
                "marginal_gain": round(high_value - low_value, 6),
                "full_baseline_value": round(full_value, 6),
            }
        )
    return rows


def expected_usage_by_scenario(dp: ResourceDP) -> List[Dict[str, object]]:
    rows = []
    for w, l in [(0, 0), (1, 0), (0, 1), (1, 1)]:
        state = game_start_state(w, l, 2, 2, 1)
        stats = simulate_policy_distribution(dp, state)
        usage = stats["expected_usage"]
        first = stats["first_usage_time_s"]
        rows.append(
            {
                "scenario": scenario_cn(scenario_key(w, l)),
                "score": f"{w}:{l}",
                "series_win_prob": round(stats["series_win_prob"], 6),
                "expected_pause": round(usage["pause"], 4),
                "expected_reset": round(usage["reset"], 4),
                "expected_repair": round(usage["repair"], 4),
                "first_pause_s": "" if first["pause"] is None else round(first["pause"], 1),
                "first_reset_s": "" if first["reset"] is None else round(first["reset"], 1),
                "first_repair_s": "" if first["repair"] is None else round(first["repair"], 1),
            }
        )
    return rows


def write_csv(path: Path, rows: List[Dict[str, object]]) -> None:
    if not rows:
        return
    with path.open("w", encoding="utf-8-sig", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)
```